MUSE: Agentic 3D Scene Authoring via Memory-Grounded Incremental Requirement Satisfaction

Ruijie Xu[1] **Xinnan Zhu**[1] **Jiayu Ying**[1] **Daoguo Dong**[2] **Yuzhou Ji**[3] **Xin Tan**[1]

[1]East China Normal University
[2]Fudan University
[3]Shanghai Jiao Tong University

Abstract

Text-driven 3D scene generation is a promising technique for digital content creation, embodied AI simulation, and interactive design, yet practical workflows often require refining, extending, or correcting existing scenes while preserving non-target content. Existing methods can produce realistic and structurally plausible scenes, but they generally lack editability with requirement-level state tracking, so part-level failures often lead to full-scene regeneration or manual intervention. To tackle this challenge, we formulate controllable 3D scene authoring as incremental requirement satisfaction, unifying construction and editing. In this paper, we present MUSE, a memory-grounded multi-agent framework in which an Architect compiles instructions into structured requirements, a Sculptor executes local scene operations, and an Inspector verifies each step while updating Working, Scene, and Skill Memory. To evaluate requirement-level controllability and preservation-aware editing, we introduce AuthorBench, offering 145 constrained construction cases and a 1,584-case preservation-aware editing pool paired with external structured checks. On full construction cases, MUSE improves All-Goal success from 37.9 to 80.7 and surface-constraint fulfillment from 35.0 to 92.6 over the strongest baseline. On a stratified 240-case editing test split, MUSE achieves 49.6 All-Goal success, 99.9 preservation rate, and only 0.6 unintended change rate. Beyond automated metrics, human evaluations on compared local-editing baselines support stronger alignment with user intent, and downstream navigation-proxy tests indicate stronger spatial stability. Combined with ablations validating our memory designs, these results establish MUSE as an effective framework for controllable 3D scene authoring.

1 Introduction

Text-driven 3D scene generation enables users to create complex three-dimensional environments from natural language, with broad applications in simulation, game design [3, 21–23, 19]. However, many practical settings, such as embodied AI training and interactive design, require more than one-shot synthesis: users often need to add, remove, or adjust local content in an existing scene while keeping non-target regions structurally and semantically stable. Supporting this workflow requires controllable generation, where a system maintains an explicit, persistent representation of the scene state and generation intent to guide subsequent local operations.

Despite substantial progress in spatial modeling, existing text-driven 3D scene generation methods still fall short of controllable generation. They can be grouped into three families. Data-driven

Preprint.

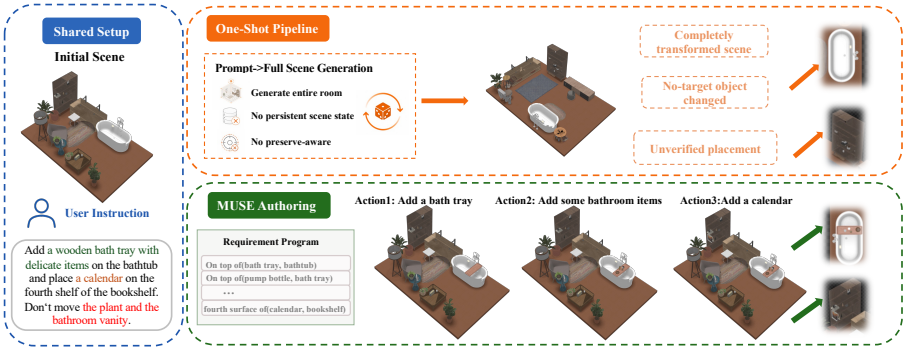


Figure 1: **From one-shot regeneration to persistent 3D scene authoring.** One-shot pipelines regenerate the whole scene and may disturb non-target regions, whereas MUSE performs verified local actions over persistent memory to preserve satisfied content.

methods [4, 2, 5, 6, 1, 14] rely on implicit generation processes, making it difficult to explicitly control local structures. LLM-based planning methods [7, 3, 8] improve open-vocabulary flexibility, but rely on one-shot context without persistently tracking execution history or unmet constraints. Agentic methods introduce tool use and closed-loop reflection [15, 17, 19], yet most of them still optimize within a single generation round rather than manage scene state across interactions. Consequently, existing systems generally lack explicit modeling and binding of requirement-level states, making it difficult to repair local failures without disturbing non-target content. In practice, when a local result is unsatisfactory, they often resort to global regeneration instead of truly controllable local revision.

Therefore, scene editability is central to controllable 3D generation. Practical workflows often do not start from an empty scene, but instead refine, extend, or correct an existing one. Accordingly, rather than regenerating globally after local failures, an ideal system should reuse semantic relations, satisfied bindings, and intermediate scene state, while applying a shared set of atomic authoring actions to update only the target region and preserve non-target content. Based on this observation, we reformulate 3D scene synthesis as a controllable authoring process that unifies generation and editing. Centered on persistent state, it supports both initial construction and fine-grained editing through requirement decomposition, local execution, verification, and memory mechanisms.

Realizing this incremental authoring process requires structured memory. *Working Memory* tracks requirement status, diagnostic signals, and protected bindings, preventing subsequent operations from breaking satisfied content. *Scene Memory* maintains a persistent hierarchical representation of the 3D environment to enable stepwise spatial reasoning. *Skill Memory* stores reusable decomposition patterns for recurring scene motifs, reducing repeated decomposition errors.

Based on this design, we present MUSE, a multi-agent framework for unified 3D scene authoring. For each instruction, the *Architect* compiles natural language into structured requirement programs, the *Sculptor* incrementally executes them, and the *Inspector* verifies each step and updates memory. This tight integration ensures MUSE reliably identifies unmet requirements, protects satisfied content, and maintains a persistent scene state across continuous interactions.

To evaluate this perspective, we introduce AuthorBench, a comprehensive benchmark for requirement-level 3D scene authoring. It features 145 constrained construction cases and a 1,584-case preservation-aware editing set, paired with structured checks to assess fine-grained spatial and semantic constraints. Experiments in construction show MUSE substantially outperforms baselines, improving All-Goal construction success from 37.9 to **80.7** and surface fulfillment from 35.0 to **92.6**. In editing, it achieves **49.6** All-Goal success with near-perfect locality (**99.9** preservation, **0.6** unintended changes). Moreover, human evaluations and downstream navigation-proxy tests support its alignment and spatial stability. Finally, ablations validate the role of verification and memory mechanisms in controllable authoring. In summary, our contributions are:

- We formulate controllable 3D scene authoring as incremental requirement satisfaction over persistent scene state, establishing a unified requirement-level foundation for construction and editing through decomposition, execution, verification, and preservation.

- We present MUSE, a memory-grounded multi-agent framework in which Architect, Sculptor, and Inspector agents operate over Working, Scene, and Skill Memory to support local revision, protection of satisfied content, and reusable decomposition of recurring scene motifs.
- We introduce AuthorBench, a unified requirement-level benchmark for evaluating both scene construction and preservation-aware editing. Extensive evaluations across automated metrics, human preference studies, downstream navigation tests, and ablations demonstrate that MUSE achieves strong performance against recent baselines in controllable 3D scene authoring.

2 Related work

Text-driven 3D scene synthesis. Prior work on text-driven 3D scene synthesis spans three main lines. Data-driven methods capture object distributions and relations from indoor scene datasets [4, 2, 5, 6, 1, 14], but offer limited explicit control over local structure and intermediate decisions. LLM-based planning methods improve open-vocabulary scene construction through spatial programs or layout optimization [7, 3, 8], while agentic frameworks introduce tool use and iterative refinement [15, 17, 19]. However, these methods are fundamentally restricted to one-shot generation from a prompt, making it difficult to isolate failed constraints, revise only the affected regions, and preserve previous results during continuous authoring. To address this issue, we introduce controllable 3D scene authoring, which unifies construction and editing at the requirement level.

Language-guided 3D scene editing. Existing language-guided 3D scene editing methods can be divided into two broad categories. Conversational or LLM-driven systems support natural-language object addition, removal, and feedback-based adjustment [28, 29], but often rely on implicit reasoning for constraint satisfaction and object protection. Structured methods improve output organization through scene-token, scene-graph, or action-sequence prediction [30, 31], yet still tend to handle complex edits as holistic commands rather than as independently verifiable requirements. In our setting, the goal is to satisfy a new instruction in a preservation-aware manner, where non-target content should remain stable. Recent planning-based editors move in this direction [18], but still do not provide a unified mechanism for tracking progress and preservation during local revision. To overcome these limitations, we formulate editing as memory-grounded incremental requirement satisfaction, leveraging structured memory to explicitly track progress and preserve non-target content.

Memory and structured state in LLM agents. Explicit memory and structured state have proven useful for long-horizon reasoning and multi-step decision making in embodied navigation, LLM agents, and scene-graph reasoning, including spatial maps, open-vocabulary scene graphs, skill libraries, memory streams, and intermediate reasoning traces [9–12, 24, 26, 27, 13]. In 3D scene synthesis, recent methods also support iterative feedback or multi-turn interaction during scene synthesis [15, 17, 19, 25]. However, systems such as SceneReVis [25] typically still rely on dialogue history or model context as their core state. This implicitly restricts their ability to track satisfied constraints, isolate preservable content, and execute safe local modifications. Our work therefore introduces task-structured memory for controllable 3D scene authoring, keeping requirement progress, preservable content, and editable scene structure explicit across interactions.

3 Method

We present MUSE, a multi-agent framework that realizes controllable 3D scene authoring through persistent requirement-level state. Given either a complex construction prompt or a targeted edit instruction, MUSE compiles the input into a structured requirement program, executes requirements locally, verifies each step, and updates memory to preserve satisfied content. Figure 2 presents the full five-step pipeline. Below, we first formulate scene authoring as incremental requirement satisfaction, then describe the Architect, Sculptor, and Inspector modules.

3.1 Task formulation

We formulate *controllable 3D scene authoring* as incremental requirement satisfaction over a persistent scene state. At interaction t, the system receives a current scene $\mathcal{S}_t$, a natural-language input I_t (either a construction prompt or a targeted edit instruction), and the memory state $\mathcal{M}_t$. This input is first compiled into a structured requirement program $\mathcal{B}_t = \{r_i\}_{i=1}^{n}$. Driven by $\mathcal{B}_t$, the system

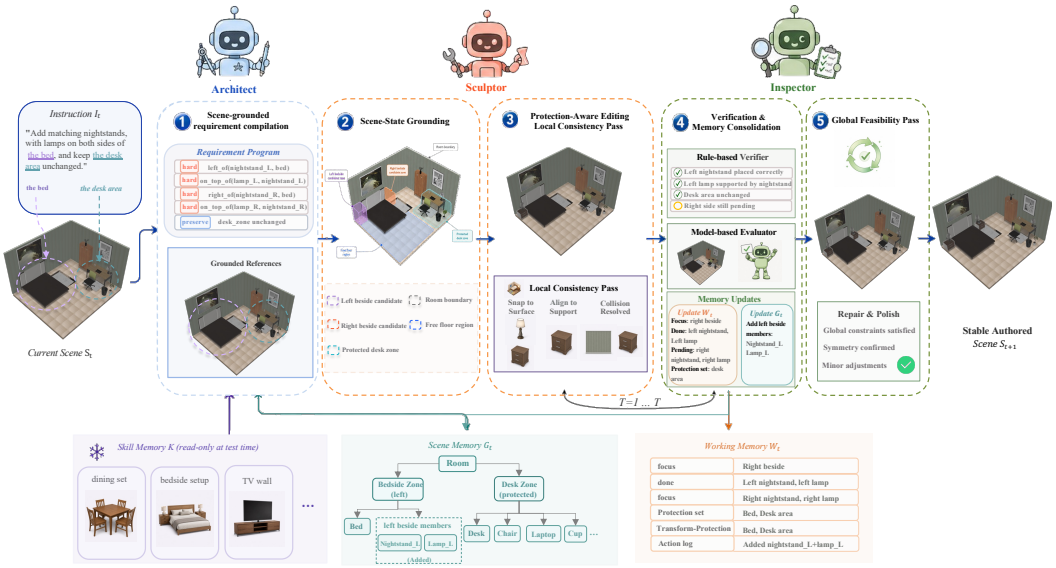


Figure 2: **Overview of MUSE.** For each interaction t, MUSE follows a five-stage authoring loop: **(1)** the Architect compiles I_t with Scene Memory $\mathcal{G}_t$ and Skill Memory $\mathcal{K}$ into a requirement program $\mathcal{B}_t$; **(2)** the Sculptor grounds $\mathcal{S}_t$ into an editable room state; **(3)** the Sculptor executes focused requirements with protection-aware local repair; **(4)** the Inspector verifies requirement satisfaction and consolidates Working and Scene Memory; and **(5)** a global feasibility pass resolves residual conflicts and returns $\mathcal{S}_{t+1}$ with updated persistent memory.

iteratively produces a sequence of atomic operations

$$\Delta_t = (\delta_1, \ldots, \delta_m), \qquad \mathcal{S}_{t+1} = \mathcal{S}_t \oplus \Delta_t, \tag{1}$$

where $\oplus$ signifies the sequential execution of these operations on the scene state. Each operation $\delta_j \in \Delta_t$ instantiates an atomic authoring action, such as object insertion, removal, transformation, or support-aware placement. The objective is **incremental requirement satisfaction**: instead of resolving the dense program simultaneously, the system iteratively schedules and fulfills requirements until every $r_i \in \mathcal{B}_t$ is verified.

To keep this incremental loop stable, the system relies on a three-layer memory state $\mathcal{M}_t = (\mathcal{W}_t,\ \mathcal{G}_t,\ \mathcal{K})$. *Working Memory ($\mathcal{W}_t$)* is instantiated per-input, storing the execution focus, satisfaction status, role bindings, and protection sets to track progress and prevent regressions. *Scene Memory ($\mathcal{G}_t$)* persists across interactions. By maintaining the room shell, support surfaces, functional zones, and the core object graph, it grounds spatial references, supports local repairs, and ensures the scene structure remains editable. Finally, *Skill Memory ($\mathcal{K}$)* acts as a reusable library of motif priors, decomposition cues, and failure warnings to improve the parsing of recurring scene configurations.

Under this formulation, construction and editing are unified as two operating modes. Construction starts from an empty or partially specified room shell and incrementally realizes requirements to build a new scene. Editing starts from an existing scene and incorporates preservation constraints into the requirement program, ensuring that new instructions can be satisfied without disturbing non-target content. Appendix A.1 provides the full executable loop and pseudocode.

Full closed-loop control flow. To make the method self-contained in the main text, Algorithm 1 spells out the execution loop used in both construction and editing. The loop differs from one-shot generation in two ways: it updates Working Memory and Scene Memory after every verified step, and it separates local consistency repair from the final global feasibility pass so that protection constraints can be enforced before broad cleanup.

3.2 Architect: scene-grounded requirement compilation

Forcing an LLM to satisfy a dense set of spatial constraints simultaneously often overloads its combinatorial reasoning, leading to dropped, hallucinated, or unverified constraints. The Architect addresses

Algorithm 1. MUSE closed-loop authoring with structured memory

Input: Scene $\mathcal{S}_t$, instruction I_t, scene memory $\mathcal{G}_t$, skill library $\mathcal{K}$, max steps $T_{\max}$
Output: Updated scene $\mathcal{S}_{t+1}$ and memory $(\varnothing, \mathcal{G}_{t+1}, \mathcal{K})$
$\mathcal{B}_t \leftarrow \textsc{Architect}(I_t,\ \mathcal{G}_t,\ \mathcal{K})$
$\mathcal{R}_0 \leftarrow \textsc{GroundSceneState}(\mathcal{S}_t,\ \mathcal{G}_t,\ \mathcal{B}_t)$
$\mathcal{W}_0 \leftarrow \textsc{InitWorkingMemory}(\mathcal{B}_t,\ \mathcal{R}_0);\quad \mathcal{G}_0 \leftarrow \mathcal{G}_t$
for $\tau = 1$ **to** $T_{\max}$ **do**
  $\text{calls}_\tau \leftarrow \textsc{Sculptor.Plan}(\mathcal{B}_t,\ \mathcal{R}_{\tau-1},\ \mathcal{W}_{\tau-1},\ \mathcal{G}_{\tau-1})$
  $\mathcal{S}_\tau \leftarrow \textsc{Sculptor.Execute}(\mathcal{S}_{\tau-1},\ \text{calls}_\tau,\ \mathcal{W}_{\tau-1})$
  $\mathcal{R}_\tau \leftarrow \textsc{LocalConsistencyPass}(\mathcal{S}_\tau,\ \mathcal{R}_{\tau-1},\ \text{calls}_\tau)$
  $\mathcal{F}_\tau \leftarrow \textsc{Inspector.Verify}(\mathcal{R}_\tau,\ \mathcal{B}_t,\ \mathcal{W}_{\tau-1})$
  $\mathcal{W}_\tau \leftarrow \textsc{UpdateWorking}(\mathcal{W}_{\tau-1},\ \text{calls}_\tau,\ \mathcal{F}_\tau)$
  $\mathcal{G}_\tau \leftarrow \textsc{UpdateScene}(\mathcal{G}_{\tau-1},\ \mathcal{R}_\tau)$
  if all key requirements are satisfied, no blocking issue remains, or a stall is detected **then break**
end for
$(\mathcal{S}_{t+1}, \mathcal{R}_{\text{final}}) \leftarrow \textsc{GlobalFeasibilityPass}(\mathcal{R}_\tau)$
$\mathcal{G}_{t+1} \leftarrow \textsc{UpdateScene}(\mathcal{G}_\tau,\ \mathcal{R}_{\text{final}})$
return $\mathcal{S}_{t+1},\ (\varnothing, \mathcal{G}_{t+1}, \mathcal{K})$

this by compiling I_t into a structured *requirement program* $\mathcal{B}_t$ whose units are typed, independently trackable, and schedulable. Rather than parsing language in isolation, it combines I_t with the Scene Memory $\mathcal{G}_t$ and retrieved decomposition priors from $\mathcal{K}$, turning a holistic instruction into verifiable units and giving the downstream loop explicit targets for execution, retry, and preservation.

Requirement program structure. The requirement program $\mathcal{B}_t = (I_t,\ \textit{room\_type},\ \{r_1, \ldots, r_n\},\ \mathbf{g},\ \mathbf{s})$ consists of atomic requirements r_i, scene-aware constraints $\mathbf{g}$, and a style profile $\mathbf{s}$. Each requirement r_i carries a type $\in \{\texttt{hard}, \texttt{edit}, \texttt{style}, \texttt{preserve}, \texttt{forbid}\}$, a priority, and an optional *checkable predicate* ϕ_i for spatial and counting assertions; requirements with ϕ_i are verified deterministically, while those without receive model-based judgment (Appendix A.2). Here, `hard` denotes required construction constraints, `edit` denotes requested modifications, `style` denotes soft appearance preferences, `preserve` protects existing content, and `forbid` encodes negative constraints. By leveraging $\mathcal{G}_t$, the Architect resolves context-dependent references before handing requirements to the Sculptor; dependency completeness is further ensured by automatic entity injection (Appendix D.1).

Requirement and feedback records. The structured requirement record used by the loop is

$$r_i = \big(\textit{type}_i,\ \textit{text}_i,\ \textit{priority}_i,\ \phi_i\big),$$

where the optional predicate has the form

$$\phi_i = \big(\textit{predicate},\ \textit{subject},\ \textit{object},\ \textit{qualifiers}\big).$$

The predicate field covers deterministically checkable spatial and counting assertions such as `count`, `exists`, `left_of`, `right_of`, `near`, `on_top_of`, and `against_wall`. After each execution step, the Inspector returns feedback records

$$f_i = \big(\textit{status}_i,\ \textit{source}_i,\ \textit{matched\_ids}_i,\ \textit{failure\_mode}_i,\ \textit{repair\_hint}_i\big),$$

where $\textit{status}_i \in \{\texttt{satisfied}, \texttt{unsatisfied}, \texttt{unknown}\}$. The $\textit{matched\_ids}_i$ field expands protection bindings when a requirement passes, while $\textit{failure\_mode}_i$ and $\textit{repair\_hint}_i$ provide targeted context for the next Sculptor step.

Skill memory $\mathcal{K}$: motif-guided decomposition. Compositional instructions such as "bedside setup" or "dining area with chairs around the table" often contain recurring structural motifs. Inspired by HSM [20], which highlights reusable hierarchical motifs in indoor scenes, we treat these recurring decomposition patterns as reusable skills for controllable 3D scene authoring rather than as one-off parsing heuristics. Skill Memory stores such motif-level priors so that frequently recurring instruction patterns can be accumulated and reused, and later decomposed into separate, checkable requirements more reliably. During evaluation, $\mathcal{K}$ is frozen to prevent data leakage; outside evaluation, new motif cards can be added without any parameter updates (Appendix D.2).

Table 1: **Structured state used by MUSE.** The main fields support progress tracking, preservation, and later-turn grounding. Working Memory is per-input; Scene Memory persists across interactions.

State	Stored fields	Role in the authoring loop
Working focus	Current requirement, pending/done partition, diagnostic issues	Selects the next local action and detects stalls
Protection state	Target set T, protected set P, transform-protected set P^T	Blocks deletion, replacement, movement, rotation, and resizing of protected content
Role bindings	Requirement-to-object matches and semantic role links	Grounds later references and expands protection after verification
Action history	Tool calls, blocked actions, degradations, repair attempts	Explains failures and conditions the next planning step
Scene graph	Stable object IDs, support edges, relation edges, wall anchors	Maintains editable structure across turns
Spatial domains	Room shell, wall domains, support surfaces, zones, free-floor regions	Grounds declarative placement and constrains local repair

3.3 Sculptor: requirement-grounded incremental execution

Incremental execution creates a regression risk: later requirements may inadvertently undo content that has already been satisfied. The Sculptor addresses this by executing one requirement at a time on an explicitly instantiated scene state while maintaining protection over already-satisfied content, enabling preservation-aware local revision.

Scene-state grounding. Spatial tools require explicit geometric domains, such as support surfaces, wall domains, and free-floor regions that are not directly available from a raw object list. Before planning begins, the Sculptor constructs an initialized room state $\mathcal{R}_0$ from $\mathcal{S}_t$ and $\mathcal{G}_t$ to register these necessary structural anchors. This grounding step allows later declarative spatial intent to be mapped stably into concrete placements and transformations.

Requirement-grounded planning. To safely execute scheduled constraints, the system must track satisfaction status, pending targets, and objects demanding protection. Working Memory $\mathcal{W}_\tau$ records focus, satisfaction status, role bindings, and diagnostic history at each inner turn τ (full schema in Appendix A.3). To prevent regressions, the Sculptor maintains two nested protection sets: the **protected set** P_τ forbids deletion or replacement of anchors, role-bound objects, and objects relied upon by satisfied requirements, while the **transform-protected set** $P_\tau^T \supseteq P_\tau$ additionally forbids moves, rotations, and resizes. After verification, these protection states are updated together with newly satisfied requirements, guaranteeing that later operations cannot compromise established progress.

Execution and local consistency pass. The Sculptor equips the planner with shared atomic authoring actions, encompassing both *declarative* tools for expressing spatial relations and *imperative* tools for direct object manipulation. The executor grounds declarative calls into geometric operations over the registered domains, allowing the planner to express spatial intent in a coordinate-free manner. Crucially, if the planner proposes an action that violates the protection sets P_τ or P_τ^T, the executor preemptively blocks it before any scene modification occurs.

After execution, an automated *local consistency pass* resolves newly introduced collisions, support-placement offsets, and minor interpenetrations within the edited neighborhood. Because both preemptive blocking and local repair are constrained by P_τ and P_τ^T, modifications remain highly localized, yielding a stable geometry for subsequent verification. More detailed tool definitions, fallback behaviors, and re-verification rules are deferred to Appendix A.7.

Tool lowering and repair scope. Declarative calls such as `place_on_support`, `attach_to_wall`, and `add_object_with_relation` are lowered into imperative placement calls only after the executor queries the relevant support surface, wall segment, or reference-object bounding box. The local consistency pass then inspects only the edited neighborhood: newly added

or moved objects, their supports, and immediate collision partners. If a repair would require moving or deleting an object in P_τ or P_τ^T, the repair is rejected and surfaced as an Inspector issue rather than silently damaging preserved content.

3.4 Inspector: compositional verification and memory consolidation

Without explicit verification, the loop lacks reliable signals for deciding whether a specific constraint has been satisfied, which action should be retried, and when execution can stop. The Inspector addresses this by performing per-requirement verification and consolidating the result into persistent memory that provides the stopping, recovery, and continuation signals for subsequent execution.

Compositional verification. Per-requirement verification requires distinguishing between constraints that can be checked deterministically and those that require model-assisted judgment. We therefore combine two complementary mechanisms. If a requirement carries a checkable predicate ϕ_i, a *rule-based verifier* performs deterministic geometric checks such as counting matched objects, evaluating spatial predicates, and checking support or wall placement. For soft or perceptual aspects, a *model-based feedback agent* judges the remaining soft or perceptual aspects from the repaired scene state and rendered views. Deterministic results take precedence whenever available, so model-based feedback complements rather than overrides verifiable constraints. The full feedback structure and reconciliation details are provided in Appendix A.2.

Memory consolidation and scheduling. Upon verification, the Inspector concurrently updates two memory layers. First, it updates Working Memory $\mathcal{W}_\tau$ with newly satisfied targets, expanding the nested protection sets (P_τ, P_τ^T) accordingly. Simultaneously, it consolidates the validated layout into Scene Memory $\mathcal{G}_\tau$, establishing a persistent structural graph (e.g., support hierarchies, zones) for future interactions. This dual-update ensures that later edits inherit a structured state rather than a raw object list. The scheduler then selects the next pending requirement, dynamically adjusting the execution order to prevent stalling on difficult constraints (Appendices A.6 and A.4).

Loop termination and global feasibility pass. The inner loop concludes once all scheduled requirements are verified or when further execution can no longer yield productive progress. Finally, the Inspector executes a *preservation-aware global feasibility pass* that resolves any residual scene-level collisions and applies lightweight stability fixes. This yields a clean, structurally sound scene state ready for the next user instruction without compromising protected non-target content.

4 AuthorBench: a compositional authoring benchmark

Existing 3D scene-synthesis benchmarks mainly evaluate holistic scene quality, such as physical plausibility, visual realism, and semantic alignment [15], but they do not reveal whether each user constraint is satisfied and rarely test realistic local editing with preservation constraints. To fill this gap, we introduce **AuthorBench**. As illustrated in Figure 3, AuthorBench shifts the evaluation paradigm from holistic scene scoring to individual requirement verification. For fairness, each case pairs a natural-language input with external structured checks: programmable constraints are evaluated by deterministic predicates, while semantic constraints are evaluated by an independent visual-semantic judge, without relying on any method-internal verifier.

Subsets. AuthorBench contains 145 construction cases and 1584 single-instruction editing cases across six room types: bedroom, living room, dining room, study, office, and kitchen. Construction starts from an empty room; editing starts from source scenes drawn from Edit-As-Act [18] and Imaginarium [16], and specifically requires preservation constraints to test explicit local modifications rather than full-scene regeneration. All experiments use a fixed 240-case stratified editing split balanced by room type and Easy/Medium/Hard/Complex tiers; Appendix B.2 gives the sampling rules and tier definitions.

Annotation and metrics. Annotation compiles natural-language requirements into typed requirement checks and then applies human review. Checks cover object, relation, surface, zone, preserve, and forbid types, enabling AuthorBench to evaluate goal completion, local preservation, and physical validity. Main experiments evaluate three dimensions: requirement satisfaction via *Goal Fulfillment*

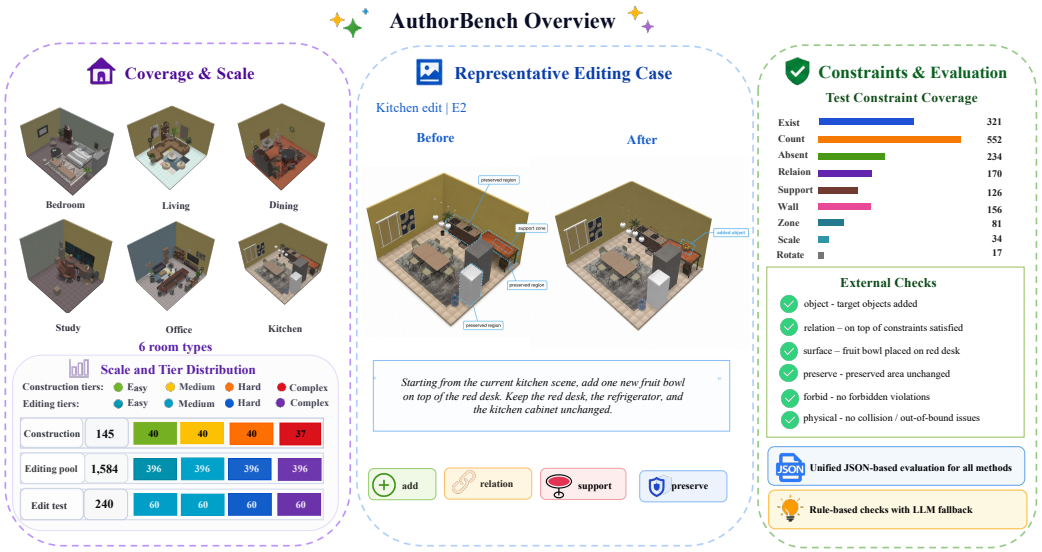


Figure 3: **AuthorBench overview.** AuthorBench unifies scene construction and preservation-aware editing under requirement-level evaluation. **(Left)** Dataset coverage across six room types and complexity tiers. **(Middle)** A representative editing case decomposing a user instruction into verifiable atomic constraints. **(Right)** The evaluation taxonomy, pairing each case with typed external checks.

Table 2: **AuthorBench coverage.** The fixed editing test split preserves the main operation families from the 1584-case pool while enforcing room-level and edit-tier balance.

(a) Edit design		
Statistic	**Pool**	**Test**
E1 atomic	396	60
E2 grounded	396	60
E3 structured	396	60
E4 compound	396	60
Imaginarium	1488	193
Edit-As-Act	96	47

(b) Scene and target burden		
Statistic	**Pool**	**Test**
S1 simple	300	35
S2 medium	1008	135
S3 dense	192	41
S4 extreme	84	29
Q0 single target	1068	161
Q1 two–three targets	374	57
Q2 four-plus targets	142	22

(GF) and *All-Goal* success; editing locality via *Preservation Rate (PR)* and *Unintended Change Rate (UCR)*; and physical validity via *Out-Of-Bounds (OOB)* and *Collisions (Col.)*.

Split construction and check coverage. The construction subset contains 145 cases across six room types: bedroom, living room, dining room, study, and office each contribute 25 cases, while kitchen contributes 20 cases. The editing benchmark contains a 1584-case instruction pool, balanced across E1 atomic, E2 grounded, E3 structured, and E4 compound edits with 396 cases per tier. The fixed 240-case editing test split is a deterministic stratified sample with 10 cases for every edit-complexity and room-type cell, yielding 40 editing cases per room type and 60 cases per edit tier. The released benchmark contains 1043 construction goal checks, 4140 editing-pool goal checks with 4752 preserve checks, and 648 goal checks with 720 preserve checks on the fixed editing test split.

External verifier inventory. AuthorBench scoring is intentionally separated from MUSE's internal verifier. Deterministic checks are applied after scene export whenever the requirement can be expressed over object IDs, bounding boxes, support surfaces, wall domains, or zones. Residual semantic and perceptual constraints are judged by an independent visual-semantic judge that receives the same prompt, exported scene representation, and rendered views for all methods.

Table 3: **External predicate inventory used by AuthorBench.** The verifier operates after scene export and does not require a method to use MUSE's internal requirement program or feedback interface.

Family	Predicates / checks	Inputs	Typical failure modes
Object	`exists`, `count`, `negative-existence`, category match	Object IDs, canonical category, count target	Missing object, extra forbidden object, wrong category
Relation	`left_of`, `right_of`, `front_of`, `behind`, `near`, `far`, `parallel_to`	Subject/object boxes, room frame, facing direction	Wrong side, too far/near, orientation mismatch
Surface	`on_top_of`, `under_support`, `wall-anchored-to`, `against_wall`	Support surface, wall domain, object footprint, height	Unsupported object, wall offset, wrong support
Zone	`inside_zone`, `zone_contains`, `free-floor-area`, clear functional region	Zone polygon, object footprint, free-space grid	Object outside zone, cluttered zone, insufficient clear area
Preserve	`preserve_existence`, `preserve_pose`, `preserve_relation`, `preserve_support`	Source-scene IDs, original poses and relations	Deleted non-target, moved anchor, broken support/relation
Forbid	`forbid_category`, `forbid_region`, `forbid_relation`	Negative constraint, scene objects, zones/relations	Forbidden object, forbidden placement, forbidden relation
Physical	OOB, collision pairs, support-height consistency	Room boundary, bounding boxes, support heights	Out-of-room object, interpenetration, floating/sunken object

5 Experiments

Our evaluation assesses *scene construction* (§5.1), *preservation-aware editing* (§5.2), and *additional analyses* covering human preference, component ablations, and downstream utility (§5.3).

Setup & Baselines. We evaluate released systems under the unified AuthorBench protocol (§4). Construction uses 145 cases starting from empty room shells; editing uses the fixed 240-case stratified split. We compare MUSE against LayoutGPT [7], Holodeck [3], LayoutVLM [8], ReSpace [30], and SceneReVis [25] for construction, and against LayoutGPT-E, Edit-As-Act [18], SceneReVis, and ReSpace-E for editing. **Metrics** strictly follow §4; results are averaged across their respective evaluation subsets.

Implementation details. For comparable API-driven adapters, we use GPT-4o and convert outputs once into the common AuthorBench format for identical rendering and rule-based evaluation. We adapt LayoutGPT and ReSpace to editing as LayoutGPT-E and ReSpace-E: LayoutGPT-E conditions on the serialized source scene, while ReSpace-E uses its native structured-scene edit pipeline. Implementation, asset, and prompt details are in Appendices B.5, D.3, and D.4.

Fairness protocol. For every baseline, we keep the released backbone, native prompting style, and stopping rule. The adapter standardizes only shared inputs and outputs: AuthorBench case definitions, room shells or source scenes, common assets when a conversion step is required, final rendering, and post-export evaluation. If a baseline emits a native layout or scene format, we convert it once into the unified AuthorBench scene JSON and score the resulting scene with the same verifier used for all methods. Failed or partial runs are counted from the last emitted scene rather than manually repaired, so the reported comparison remains an end-to-end comparison under a shared benchmark protocol.

5.1 Scene Construction

Table 4 reports quantitative construction performance. MUSE is the only framework to achieve an *All-Goal* success rate above 80% (80.7), significantly outperforming the strongest one-shot baseline, LayoutGPT (37.9). When analyzing individual constraint families, the primary performance gap emerges in support-surface relations: MUSE fulfills 92.6% of surface constraints, whereas baseline performance drops below 35.0%. Furthermore, MUSE demonstrates consistent robustness across instruction difficulty. As evaluated by the complexity retention metric (Ret.), MUSE retains 96.6%

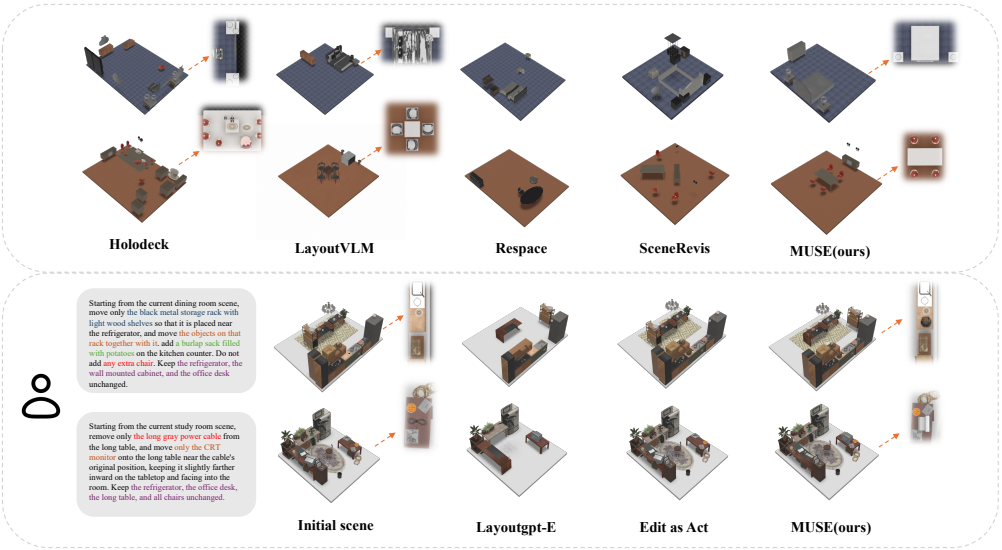


Figure 4: **Qualitative construction and editing comparisons.** Compared to baselines, MUSE generates more organized spatial layouts in scene construction, and performs precise local edits without breaking the original scene structure.

Table 4: **Main construction results on AuthorBench.** We report requirement fulfillment, All-Goal success, complexity robustness (Ret. = Complex/Easy), and physical validity (OOB = out-of-boundary objects, Col. = colliding object pairs). Macro averages object, relation, surface, and zone fulfillment; Core averages object and relation fulfillment.

Method	Goal Fulfillment ↑							All-Goal↑	Complexity Robustness ↑					Physical Validity ↓	
	Obj.	Rel.	Surf.	Zone	Macro	Macro*	Core		Easy	Med.	Hard	Complex	Ret.	OOB	Col.
LayoutGPT [7]	98.4	83.8	35.0	79.0	74.1	87.1	91.1	37.9	99.3	86.0	83.4	82.2	82.7	1.4	2.3
Holodeck [3]	83.3	57.6	21.5	30.9	48.3	57.3	70.5	14.5	84.7	73.7	67.9	54.1	63.9	0.0	0.0
LayoutVLM	98.0	69.7	22.1	59.3	62.3	75.7	83.9	26.9	100.0	77.4	72.7	81.7	81.7	0.0	0.0
ReSpace [30]	71.9	36.4	8.6	32.1	37.3	46.8	54.2	11.0	81.7	59.6	50.8	47.0	57.5	0.4	0.5
SceneReVis [25]	46.9	17.2	9.2	8.6	20.5	24.2	32.1	2.1	47.8	40.6	32.0	31.4	65.6	0.3	0.9
MUSE	**99.3**	**93.9**	**92.6**	**82.7**	**92.1**	**92.0**	**96.6**	**80.7**	99.3	**97.9**	**94.9**	**95.9**	**96.6**	**0.0**	0.1

of its Easy-tier performance when evaluated on dense Complex-tier cases, while simultaneously avoiding physical errors (OOB 0.0, Col. 0.1).

Qualitatively, Figure 4(a) reflects these numerical differences. One-shot baselines often retrieve correct object categories but struggle with spatial dependencies: support objects are separated from their intended surfaces, and multi-object functional groups (e.g., dining sets) drift from the requested room-level arrangement. In contrast, MUSE preserves explicit global layout structures and grounds target objects precisely to their assigned surfaces.

5.2 Preservation-Aware Scene Editing

A practical editor must satisfy new instructions without destroying unmentioned background. Table 5 reports quantitative editing performance, revealing a clear completion-locality trade-off among baselines. Prompt-based regenerators (LayoutGPT-E) achieve high object fulfillment (86.0%) but effectively overwrite the source scene (PR 25.1%, UCR 95.7%). Conversely, methods attempting local updates (Edit-As-Act, ReSpace-E, SceneReVis) execute only partial edits with redundant actions, yielding *All-Goal* success below 12%. MUSE mitigates this trade-off, achieving the highest task completion (Macro 69.0, All-Goal 49.6) while maintaining strict locality (PR 99.9%, UCR 0.6%) and near-zero edit-induced physical errors.

Appendix Figure 7 visualizes this trade-off. Baselines cluster either in high-damage regions (using rewriting as a shortcut) or low-completion regions (failing complex spatial logic). MUSE is positioned in the preferred top-left region, improving fulfillment without sacrificing stability. Figure 4(b)

qualitatively confirms this precise control. Given a compound instruction to move a rack, transfer its objects, and add a sack while protecting cabinets, LayoutGPT-E scrambles the layout, and Edit-As-Act fails the object transfer. MUSE confines modifications to targets, executing the coupled edit while largely preserving the protected context.

Table 5: **Main editing results on AuthorBench.** We report fulfillment, All-Goal success, robustness, edit locality (PR/UCR), and edit-induced errors.

Method	Goal Fulfillment ↑				All-Goal↑	Complexity Robustness ↑					Edit Locality		Physical Validity ↓	
	Obj.	Rel.	Surf.	Macro		Easy	Med.	Hard	Complex	Ret.	PR↑	UCR↓	OOB	Col.
LayoutGPT-E [7]	86.0	21.1	19.3	42.2	36.2	86.7	59.2	61.0	67.2	77.5	25.1	95.7	1.8	5.2
Edit-As-Act [18]	30.6	14.1	5.0	16.6	5.4	21.7	23.3	27.2	22.2	102.3	84.9	17.8	0.0	0.0
SceneReVis [25]	33.6	28.2	4.2	22.0	10.4	31.7	33.3	26.6	25.4	80.1	92.6	5.9	0.8	2.1
ReSpace-E [30]	46.3	19.7	5.9	24.0	11.2	40.8	27.5	36.2	38.6	94.6	100.0	1.9	1.2	3.5
MUSE	76.0	**47.9**	**83.2**	**69.0**	**49.6**	73.3	**71.7**	**68.6**	**80.2**	**109.4**	**99.9**	**0.6**	**0.1**	**0.3**

5.3 Additional Analyses

Human evaluation. To complement automated metrics, we invited 50 participants from diverse backgrounds to assess preservation-aware editing (details in Appendix B.6). Table 7 compares MUSE against Edit-As-Act, ReSpace-E, and SceneReVis. Raters evaluated instruction satisfaction, preservation, physical plausibility, and provided a 4-way overall preference. MUSE achieves the highest overall preference rate. Crucially, participants consistently penalized methods that introduced unnecessary background modifications. This alignment between human choices and our automated PR/UCR metrics indicates that MUSE's localized updates better match the evaluated user intent.

Component ablation. Removing per-step verification triggers a large drop in editing completion (Macro 69.0 $\rightarrow$ 18.5). Table 6 isolates each framework component and shows that memory modules provide distinct causal benefits: protection sets govern locality (removal drops PR to 90.2% and raises UCR to 9.7%), while Scene Memory maintains structural grounding. Notably, Skill Memory is critical for complex compositional reasoning; removing it disproportionately degrades Complex-tier performance, lowering the overall editing Macro to 61.4.

Table 6: **Component ablation.** Rows remove one component from MUSE; metrics use the fixed 240-case split.

	Construction				Editing			Phys.
Variant	Macro-GF↑	Hard↑	Complex↑	HCS↑	Macro↑	PR↑	UCR↓	Col.↓
MUSE (full)	**92.2**	**95.0**	**95.9**	**95.3**	**69.0**	**99.9**	**0.6**	**0.3**
w/o focus scheduler	90.9	92.6	93.7	93.8	66.4	97.5	2.5	0.4
w/o protection sets	91.0	93.2	94.4	94.1	65.3	90.2	9.7	0.5
w/o Scene Memory	89.4	91.2	92.1	92.4	63.2	95.3	4.2	0.5
w/o Skill Memory	88.7	92.4	89.6	91.8	61.4	96.5	3.4	0.4
single-pass	14.5	26.8	25.5	34.4	18.5	76.2	19.8	0.6

Downstream utility. Table 8 evaluates 100 three-stage navigation curricula. While both incremental MUSE and stage-wise regeneration produce navigable final scenes (high reachable ratios, zero collisions), MUSE yields substantially stronger spatial continuity. It achieves higher difficulty monotonicity (0.850 vs. 0.394) and free-space preservation (0.978 vs. 0.649). This confirms that incremental authoring provides not merely static navigability, but a stable, predictable spatial substrate useful for navigation-oriented embodied-AI settings.

Table 7: **Human evaluation.** Mean ratings (1–5) and 4-way overall preference (%).

Method	Satis.↑	Preserv.↑	Plaus.↑	Pref.↑
Edit-As-Act	2.5	4.1	4.3	16.2%
SceneReVis	3.2	3.5	3.8	11.0%
ReSpace-E	3.6	3.8	3.5	9.4%
MUSE (Ours)	**4.5**	**4.8**	**4.6**	**63.4%**

Table 8: **Downstream curricula.** 300-case navigation with Mono./FS-IoU consistency metrics.

Method	Mono.↑	FS-IoU↑	Walk@3	Reach@3	Path@3	Col./OOB
Stage-wise	0.394	0.649	**0.878**	**0.999**	2.183	0.0/0.0
MUSE	**0.850**	**0.978**	0.868	0.992	**1.736**	0.0/0.0

6 Conclusion

In this paper, we formulated controllable 3D scene authoring as incremental requirement satisfaction and introduced MUSE, a memory-grounded multi-agent framework for requirement compilation, iterative execution, and step-wise verification. Together with AuthorBench, a requirement-level benchmark for construction and preservation-aware editing, our experiments show that MUSE outperforms one-shot baselines in dense construction and mitigates the editing completion-locality trade-off, achieving strong task completion with near-perfect structural stability (PR 99.9%, UCR 0.6%). Human evaluations and downstream navigation-proxy tests further support its alignment with user intent and spatial stability.

Limitation. MUSE and AuthorBench are currently limited to indoor scenes and single-instruction edits, and MUSE still relies on repeated LLM-guided verification. Future work will extend the domain scope, distill verification into efficient models, and support long-horizon multi-turn authoring. These results point toward a shift from single-shot holistic generation to verifiable, requirement-level 3D scene progression.

References

[1] Tang, J., Nießner, M., & Dai, A. (2024). DiffuScene: Denoising diffusion models for generative indoor scene synthesis. In *Proc. CVPR*.

[2] Paschalidou, D., Kar, A., Shugrina, M., Kreis, K., Geiger, A., & Fidler, S. (2021). ATISS: Autoregressive transformers for indoor scene synthesis. In *Proc. NeurIPS*.

[3] Yang, Y., Fan, R., Xu, J., Yu, M., Akkaynak, D., Gao, R., & Ilie, A. (2024). Holodeck: Language guided generation of 3D embodied AI environments. In *Proc. CVPR*.

[4] Fu, H., Cai, B., Gao, L., Zhang, L. X., Wang, J., Li, C., Zeng, Q., Sun, C., Jia, R., Zhao, B., & Zhang, L. (2021). 3D-FRONT: 3D furnished rooms with layouTs and semantics. In *Proc. ICCV*.

[5] Wang, X., Yeshwanth, C., & Nießner, M. (2021). SceneFormer: Indoor scene generation with transformers. In *Proc. 3DV*.

[6] Dhamo, H., Manhardt, F., Navab, N., & Tombari, F. (2021). Graph-to-3D: End-to-End Generation and Manipulation of 3D Scenes Using Scene Graphs. In *Proc. ICCV*.

[7] Feng, W., Zhu, W., Fu, T.-J., Jampani, V., Akula, A. R., He, X., Basu, S., Wang, X. E., & Wang, W. Y. (2023). LayoutGPT: Compositional Visual Planning and Generation with Large Language Models. In *Proc. NeurIPS*.

[8] Sun, F. Y., Liu, W., Gu, S., Lim, D., Bhat, G., Tombari, F., Li, M., Haber, N., & Wu, J. (2025). LayoutVLM: Differentiable Optimization of 3D Layout via Vision-Language Models. In *Proc. CVPR*.

[9] Chaplot, D. S., Gandhi, D., Gupta, S., Gupta, A., & Salakhutdinov, R. (2020). Learning to explore using active neural SLAM. In *Proc. ICLR*.

[10] Gu, Q., Kuwajerwala, A., Jatavallabhula, K. M., Sen, B., Agarwal, A., Rivera, C., Paul, W., Chellappa, R., Gan, C., de Melo, C. M., Tenenbaum, J. B., Torralba, A., Shkurti, F., & Paull, L. (2024). ConceptGraphs: Open-Vocabulary 3D Scene Graphs for Perception and Planning. In *Proc. ICRA*.

[11] Wang, G., Xie, Y., Jiang, Y., Mandlekar, A., Xiao, C., Zhu, Y., Fan, L., & Anandkumar, A. (2023). Voyager: An open-ended embodied agent with large language models. *arXiv preprint*.

[12] Park, J. S., O'Brien, J. C., Cai, C. J., Morris, M. R., Liang, P., & Bernstein, M. S. (2023). Generative agents: Interactive simulacra of human behavior. In *Proc. UIST*.

[13] Sumers, T. R., Yao, S., Narasimhan, K., & Griffiths, T. L. (2023). Cognitive architectures for language agents. *arXiv preprint arXiv:2309.02427*.

[14] Lin, C., Xu, J., Zhu, J., Liang, Y., Zhang, L., & Huang, S. (2024). InstructScene: Instruction-driven 3D indoor scene synthesis with semantic graph prior. In *Proc. ICLR*.

[15] Yang, Y., Jia, B., Zhang, S., & Huang, S. (2025). SceneWeaver: All-in-One 3D Scene Synthesis with an Extensible and Self-Reflective Agent. In *Proc. NeurIPS*.

[16] Zhu, X., Huang, X., Xie, Q., Deng, Z., Yu, J., Guan, Y., Liu, Z., Zhu, L., Zhao, Q., Liu, L., & Zeng, L. (2025). Imaginarium: Vision-Guided High-Quality 3D Scene Layout Generation. *ACM Transactions on Graphics*, 44(6).

[17] Pfaff, N., Cohn, T., Zakharov, S., Cory, R., & Tedrake, R. (2026). SceneSmith: Agentic Generation of Simulation-Ready Indoor Scenes. *arXiv preprint arXiv:2602.09153*.

[18] Noh, S., Seo, S., Park, G.-M., & Kang, H. (2026). Edit-As-Act: Goal-Regressive Planning for Open-Vocabulary 3D Indoor Scene Editing. In *Proc. CVPR*.

[19] Xia, H., Li, X., Li, Z., Ma, Q., Xu, J., Liu, M.-Y., Cui, Y., Lin, T.-Y., Ma, W.-C., Wang, S., Song, S., & Wei, F. (2026). SAGE: Scalable Agentic 3D Scene Generation for Embodied AI. *arXiv preprint arXiv:2602.10116*.

[20] Pun, H. I. D., Tam, H. I., Wang, A. T., Huo, X., Chang, A. X., & Savva, M. (2026). HSM: Hierarchical Scene Motifs for Multi-Scale Indoor Scene Generation. In *Proc. 3DV*.

[21] Yang, Y., Jia, B., Zhi, P., & Huang, S. (2024). PhyScene: Physically Interactable 3D Scene Synthesis for Embodied AI. In *Proc. CVPR*.

[22] Fu, R., Wen, Z., Liu, Z., & Sridhar, S. (2024). AnyHome: Open-vocabulary generation of structured and textured 3D homes. In *Proc. ECCV*.

[23] Gu, Z., Cui, Y., Li, Z., Wei, F., Ge, Y., Gu, J., Liu, M.-Y., Davis, A., & Ding, Y. (2025). ArtiScene: Language-driven artistic 3D scene generation through image intermediary. In *Proc. CVPR*.

[24] Shinn, N., Cassano, F., Gopinath, A., Narasimhan, K., & Yao, S. (2023). Reflexion: Language agents with verbal reinforcement learning. In *Proc. NeurIPS*.

[25] Zhao, Y., Sun, S., Zhang, M., Shi, Y., Yang, X., & Bian, J. (2026). SceneReVis: A self-reflective vision-grounded framework for 3D indoor scene synthesis via multi-turn RL. *arXiv preprint arXiv:2602.09432*.

[26] Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., & Cao, Y. (2023). ReAct: Synergizing reasoning and acting in language models. In *Proc. ICLR*.

[27] Huang, W., Xia, F., Xiao, T., Chan, H., Liang, J., Florence, P., Zeng, A., Tompson, J., Mordatch, I., Chebotar, Y., Sermanet, P., Brown, T., Jackson, T., Luu, L., Levine, S., Hausman, K., & Ichter, B. (2022). Inner monologue: Embodied reasoning through planning with language models. In *Proc. CoRL*.

[28] Yang, Y., Lu, J., Zhao, Z., Luo, Z., Yu, J. J. Q., Sanchez, V., & Zheng, F. (2024). LLplace: The 3D Indoor Scene Layout Generation and Editing via Large Language Model. *arXiv preprint arXiv:2406.03866*.

[29] Wang, C., Zhong, H., Chai, M., He, M., Chen, D., & Liao, J. (2024). Chat2Layout: Interactive 3D Furniture Layout with a Multimodal LLM. *arXiv preprint arXiv:2407.21333*.

[30] Bucher, M. J. J., & Armeni, I. (2025). ReSpace: Text-Driven 3D Scene Synthesis and Editing with Preference Alignment. *arXiv preprint arXiv:2506.02459*.

[31] Zhen, H., Li, X., Zhao, Y., Zhang, H., Liu, S., Mo, K., Gan, C., & Radhakrishnan, S. (2026). 3D-Layout-R1: Structured Reasoning for Language-Instructed Spatial Editing. *arXiv preprint arXiv:2603.22279*.

Appendix

The appendix is organized into five parts. Appendix A gives method and algorithm details. Appendix B documents AuthorBench and the evaluation protocol. Appendix C provides additional experimental results. Appendix D covers implementation, assets, prompts, and the frontend. Appendix E discusses limitations and broader impacts.

A Method and Algorithm Details

This part expands the closed-loop authoring process, structured state, scheduling logic, and execution machinery that are summarized in the main paper.

A.1 Closed-loop execution algorithm

This section expands the method with the full closed-loop algorithm and the supporting implementation details that are abbreviated in the main text. In particular, it clarifies how requirement execution, memory updates, consistency repair, and verifier roles fit together under the final evaluation protocol. Algorithm 2 gives the exact turn-level control flow used in all experiments.

Algorithm 2. MUSE: Incremental scene authoring with three-layer memory

Input: Scene $\mathcal{S}_t$, input I_t, scene memory $\mathcal{G}_t$, skill library $\mathcal{K}$, max steps $T_{\max}$
Output: Updated scene $\mathcal{S}_{t+1}$, updated memory $(\varnothing, \mathcal{G}_{t+1}, \mathcal{K})$
$\mathcal{B}_t \leftarrow$ ARCHITECT(I_t, $\mathcal{G}_t$, $\mathcal{K}$)
$\mathcal{R}_0 \leftarrow$ GROUNDSCENESTATE($\mathcal{S}_t$, $\mathcal{G}_t$, $\mathcal{B}_t$)
$\mathcal{W}_0 \leftarrow$ INITWORKINGMEMORY($\mathcal{B}_t$, $\mathcal{R}_0$)
$\mathcal{G}_0 \leftarrow \mathcal{G}_t$
for $\tau = 1$ **to** $T_{\max}$ **do**
  $\text{calls}_\tau \leftarrow$ SCULPTOR.PLAN($\mathcal{B}_t$, $\mathcal{R}_{\tau-1}$, $\mathcal{W}_{\tau-1}$, $\mathcal{G}_{\tau-1}$)
  $\mathcal{S}_\tau \leftarrow$ SCULPTOR.EXECUTE($\mathcal{S}_{\tau-1}$, calls_τ, $\mathcal{W}_{\tau-1}$)
  $\mathcal{R}_\tau \leftarrow$ LOCALCONSISTENCYPASS($\mathcal{S}_\tau$, $\mathcal{R}_{\tau-1}$, calls_τ)
  $\mathcal{F}_\tau \leftarrow$ INSPECTOR.VERIFY($\mathcal{R}_\tau$, $\mathcal{B}_t$, $\mathcal{W}_{\tau-1}$)
  $\mathcal{W}_\tau \leftarrow$ UPDATEWORKING($\mathcal{W}_{\tau-1}$, calls_τ, $\mathcal{F}_\tau$)
  $\mathcal{G}_\tau \leftarrow$ UPDATESCENE($\mathcal{G}_{\tau-1}$, $\mathcal{R}_\tau$)
  if (ALLKEYSATISFIED($\mathcal{F}_\tau$) $\wedge$ NOBLOCKINGISSUES($\mathcal{F}_\tau$)) $\vee$ STALLDETECTED($\mathcal{W}_\tau$) **then**
    break
  end if
end for
$(\mathcal{S}_{t+1}, \mathcal{R}_{\text{final}}) \leftarrow$ GLOBALFEASIBILITYPASS($\mathcal{R}_\tau$)
$\mathcal{G}_{t+1} \leftarrow$ UPDATESCENE($\mathcal{G}_\tau$, $\mathcal{R}_{\text{final}}$)
return $\mathcal{S}_{t+1}$, $(\varnothing, \mathcal{G}_{t+1}, \mathcal{K})$

A.2 Requirement program and feedback records

The requirement program is defined as:

$$\mathcal{B}_t = \big(I_t,\ \textit{room\_type},\ \{r_1, \ldots, r_n\},\ \mathbf{g},\ \mathbf{s}\big),$$

where $\mathbf{g}$ denotes scene-aware constraints grounded in the current room context, and $\mathbf{s}$ a style profile. Each requirement r_i is a structured record:

$$r_i = \big(\textit{type}_i,\ \textit{text}_i,\ \textit{priority}_i,\ \phi_i\big),$$

with $\textit{type}_i \in \{\texttt{hard}, \texttt{edit}, \texttt{style}, \texttt{preserve}, \texttt{forbid}\}$. The optional checkable predicate ϕ_i has the form:

$$\phi_i = \big(\textit{predicate},\ \textit{subject},\ \textit{object},\ \textit{qualifiers}\big),$$

where *predicate* $\in \{\texttt{count}, \texttt{exists}, \texttt{left\_of}, \texttt{right\_of}, \texttt{near}, \texttt{on\_top\_of}, \texttt{against\_wall}, \ldots\}$ defines a deterministically checkable spatial or counting assertion, and *subject*/*object* are typed references carrying `semantic_type`, `canonical_type`, and `role` fields for robust matching.

The Inspector returns a structured feedback record

$$\mathcal{F}_\tau = \{f_i\}_{i=1}^{n}, \qquad f_i = \big(\textit{status}_i,\ \textit{source}_i,\ \textit{matched\_ids}_i,\ \textit{failure\_mode}_i,\ \textit{repair\_hint}_i\big),$$

Table 9: **Working Memory schema.** Working Memory is reinitialized for each input and stores the state needed to make incremental execution auditable and preservation-aware.

Field	Updated by	Stores	Used for
`focus`	Scheduler	Current requirement ID and scope	Select local action target
`done/pending`	Inspector	Requirement status partition	Stop condition and retry planning
`target set` T	Architect/Scheduler	Objects intended to change	Separate target edits from non-target content
`protected set` P	Inspector	Anchors and satisfied bindings	Block deletion or replacement regressions
`transform-protected` P^T	Inspector	Objects that also cannot move/resize/rotate	Preserve layout and relation stability
`bindings`	Inspector	Role-to-object links	Ground references and expand protection
`actions`	Sculptor	Tool calls, blocked actions, degradations	Diagnose execution failures
`issues`	Inspector	Failure modes and repair hints	Guide the next focused action

where $\textit{status}_i \in \{\texttt{satisfied}, \texttt{unsatisfied}, \texttt{unknown}\}$ and $\textit{source}_i$ records whether the decision came from a rule-based predicate, model-assisted judgment, or both. When a rule-based predicate is available, its result takes precedence for the corresponding hard spatial or counting assertion; model-assisted feedback is used for residual style, perceptual, or open-ended semantic aspects. The reconciled record updates Working Memory by moving satisfied requirements to $\textit{done}_\tau$, attaching matched object IDs to protection bindings, and recording failure modes or repair hints for the next planning turn.

A.3 Working Memory and protection state

Working Memory $\mathcal{W}_\tau$ at inner turn τ maintains the full execution state:

$$\mathcal{W}_\tau = \big(\textit{focus}_\tau,\ \textit{done}_\tau,\ \textit{pending}_\tau,\ \Pi_\tau,\ \textit{bindings}_\tau,\ \textit{actions}_\tau,\ \textit{issues}_\tau\big),$$

where:

- $\textit{focus}_\tau$: the current requirement being addressed;
- $\textit{done}_\tau/\textit{pending}_\tau$: partition of the requirement set by satisfaction status;
- $\Pi_\tau = (T_\tau, P_\tau, P_\tau^T)$: the protection state (target, protected, transform-protected sets);
- $\textit{bindings}_\tau$: maps semantic roles (e.g., `bed_anchor`) to scene object IDs;
- $\textit{actions}_\tau$: log of tool calls, blocked reasons, and degradations;
- $\textit{issues}_\tau$: diagnostic messages from the most recent verification.

A.4 Scene Memory organization and bookkeeping

Scene Memory $\mathcal{G}_t$ stores the persistent spatial state needed to ground later instructions. It contains the room shell, wall domains and openings, support surfaces, free-floor regions, functional zones, stable object IDs, and object-relation edges such as support, adjacency, wall anchoring, and zone membership. After each Inspector update, satisfied requirement bindings are reflected in this graph so that later turns can resolve references such as "the desk on the east wall" or "the objects in the reading corner" against the current authored scene rather than the original input list.

Zone-level bookkeeping groups objects and requirements by functional or spatial scope. These groups are used by the focus scheduler for locality-aware ordering, by the local consistency pass to restrict repairs to the edited neighborhood, and by subsequent interaction turns to preserve non-target regions. Scene Memory is therefore the cross-turn state, while Working Memory remains per-input and is reinitialized from $\mathcal{G}_t$ and the current requirement program.

A.5 Cross-turn memory inheritance and conflict handling

Working Memory $\mathcal{W}_\tau$ is an intra-turn structure and does not persist across user turns. At the beginning of a new instruction, MUSE reinitializes working memory from the current scene state and recompiles the new requirement program. What persists is scene memory $\mathcal{G}_{t+1}$: stable object IDs, the updated

Table 10: **Scene Memory schema.** Scene Memory is the persistent state inherited across instructions; it stores editable scene structure rather than only raw object lists.

Component	Stored state	Role in later turns
Room shell	Boundary, floor frame, canonical axes	Keep placement and OOB checks consistent
Wall domains	Wall IDs, direction labels, openings, available segments	Ground "east wall" and wall-mounted edits
Support surfaces	Desks, shelves, tables, counters, surface extents/heights	Resolve support placement and repair floating objects
Free-floor regions	Occupancy/free-space summaries and clear regions	Find fallback placement and preserve navigation space
Functional zones	Zone polygons, member objects, zone-level requirements	Maintain locality and grouped edits
Object graph	Stable IDs, categories, poses, sizes, support/relation edges	Ground references and detect changed non-target objects
Satisfied bindings	Requirement-to-object matches and protected anchors	Preserve previously satisfied content

room graph, support and wall domains, zone annotations, and any grounded state needed to interpret the next instruction.

This design means that contradictory later edits are handled by recompiling against the current scene rather than enforcing a global monotonicity rule. For example, if one turn adds a sofa near the window and the next turn requests removing that sofa, the second turn simply targets the grounded sofa instance in the updated graph and removes it. The preservation guarantee therefore applies within a single authoring turn through the protection sets and verifier loop; across turns, MUSE follows the latest user instruction while retaining only the scene state needed for grounded continuation.

A.6 Focus scheduling and stall recovery

The focus scheduler selects the next requirement to address at each inner turn. Rather than a simple priority queue, it uses a multi-signal scoring function to exploit spatial locality and respect dependency ordering.

A.6.1 Scoring signals

For each candidate requirement $r_i \in \textit{pending}_\tau$, the scheduler computes a composite score from the following signals:

1. **Verifiability bonus** (+100): Requirements with rule-based verifier support (*verification_mode* = `rule_based`) receive a large bonus, as their satisfaction can be deterministically confirmed.
2. **Constraint kind priority**: Entity requirements (+35) are prioritized over edit (+45), placement (+10), and layout (-15), reflecting the dependency ordering principle: objects must exist before they can be positioned.
3. **Scope locality** (+30–45): If r_i shares spatial scope (same zone) with the previous focus requirement, it receives a locality bonus. Processing co-located requirements consecutively reduces planner context switching.
4. **Scope transition penalty**: Requirements that would cause a zone-level scope transition are penalized relative to those in the same scope, encouraging spatial coherence in the editing sequence.
5. **Priority weight**: Higher-priority requirements (lower *priority* integer) receive a modest bonus ($\max(0, 20 - \textit{priority} \times 5)$).
6. **Self-repetition penalty** (-50): If r_i was the previous focus and is still unsatisfied, it is penalized to encourage the scheduler to try alternative requirements.

A.6.2 Selection with tie-breaking

Candidates are ranked by descending score. If the top candidate's lead exceeds a threshold (default: 25 points), it is selected deterministically. Otherwise, the top-3 candidates are shortlisted and one is

Table 11: **Focus scheduler scoring signals.** The scoring terms make dependency ordering, verifiability, and spatial locality explicit rather than relying on prompt order alone.

Signal	Score	Purpose	Failure reduced
Rule-based verifiability	$+100$	Prefer requirements with deterministic feedback	Uncheckable progress claims
Edit/entity priority	$+45/+35$	Process requested edits and missing anchors early	Dangling references
Placement priority	$+10$	Schedule spatial placement after entities exist	Premature relation placement
Layout penalty	-15	Delay broad layout moves unless needed	Unnecessary scene-wide changes
Scope locality	$+30$–$+45$	Continue within the same zone or support neighborhood	Context switching and scattered edits
Priority weight	up to $+20$	Respect annotated requirement importance	Low-priority work blocking key goals
Self-repetition	-50	Avoid retrying the same failed focus indefinitely	Local stalls

sampled with probability proportional to score, using a deterministic seed derived from the turn index and candidate IDs to ensure reproducibility.

A.6.3 Progress monitoring and escape

The scheduler tracks each requirement's "ineffective focus count"—the number of consecutive turns where it was the focus but made no (or negative) progress. If this count exceeds the stall threshold (default k=3), the requirement is temporarily marked as *blocked* and deprioritized. Blocked requirements are re-eligible once other requirements have been processed, or if no unblocked candidates remain. We keep the same k=3 threshold for all experiments and do not retune it per benchmark split.

A.6.4 Dependency eligibility

A requirement r_i with `depends_on_requirement_ids` is only eligible for focus if all its dependencies are in $done_\tau$. This prevents the scheduler from selecting a placement requirement before its prerequisite entity requirement has been satisfied.

A.7 Tool hierarchy, lowering, and consistency passes

A.7.1 Tool vocabulary

The Sculptor's action vocabulary consists of two layers.

Declarative tools express spatial intent without requiring exact coordinates:

- `add_object_with_relation(ref_uid, relation, category, ...)`: Place a new object relative to an existing reference (e.g., `left_of`, `right_of`, `near`).
- `place_on_support(support_uid, category, ...)`: Place an object on a support surface (desk, nightstand, shelf).
- `place_under_support(support_uid, category, ...)`: Place an object beneath a support surface.
- `attach_to_wall(wall_direction, category, ...)`: Mount an object on a specified wall.
- `reposition_object_by_relation(uid, ref_uid, relation)`: Move an existing object to a new position defined by a spatial relation to a reference.

Imperative tools specify precise parameters:

- `add_object(category, pos, rot, size)`: Add an object at exact coordinates.
- `remove_object(uid)`: Remove an object by ID.
- `move_object(uid, new_pos)`: Move to exact position.

Table 12: **Tool hierarchy and fallback behavior.** Declarative tools expose high-level authoring intent; lowering converts them into executable geometric operations and records fallback events when exact grounding fails.

Declarative intent	Lowered operation	Fallback	Verifier affected
Relation placement	Add/move object at relation-derived floor position	Nearby free-floor placement around the reference	Relation, collision, OOB
Support placement	Add object at support-surface height and free support footprint	Nearby floor placement with logged degradation	Surface/support, collision
Wall attachment	Add object flush to wall segment and align rotation	Nearest available wall segment	Wall, orientation, OOB
Object replacement	Remove target and add replacement at inherited pose/scale	Keep target and report blocked replacement if protected	Object, preserve, forbid
Local repair	Move only edited neighborhood by minimum feasible offset	Surface issue to Inspector if protected content is required	Physical validity, previously satisfied checks

- `rotate_object(uid, new_rot)`: Set exact rotation.
- `scale_object(uid, new_size)`: Set exact dimensions.
- `replace_object(uid, new_category, ...)`: Swap category while preserving position.
- `terminate`: Signal loop termination.

A.7.2 Intent grounding (declarative-to-geometric compilation)

The executor *compiles* each declarative tool call into one or more imperative operations through domain-specific geometric reasoning:

Support domain. For `place_on_support`, the executor queries the support surface's bounding box, computes available regions on the surface (accounting for existing objects), selects a placement position, and emits an `add_object` call with the computed coordinates and a y-position equal to the surface height plus half the object height.

Wall domain. For `attach_to_wall`, the executor identifies the target wall segment, computes the wall-normal offset, and emits an `add_object` call with position flush against the wall and rotation aligned to the wall direction.

Relational domain. For `add_object_with_relation`, the executor resolves the spatial relation (e.g., `left_of` with a configurable offset) relative to the reference object's bounding box and facing direction, then emits an `add_object` call.

A.7.3 Tool degradation

When a declarative tool's geometric computation fails (e.g., no available space on a support surface, or the reference object cannot be found), the system applies a graceful degradation strategy:

1. **Support fallback**: If `place_on_support` fails, degrade to `add_object` with a free-floor position near the support.
2. **Relation fallback**: If `add_object_with_relation` fails due to collision, degrade to `add_object` at a nearby floor position.
3. **Wall fallback**: If `attach_to_wall` fails, degrade to `add_object` placed against the nearest available wall segment.

Each degradation event is logged as a `ToolDegradation` record (containing the original tool, the fallback tool, and the reason) and reported in the working memory's action log, enabling the Inspector and subsequent planning turns to be aware of the compromise.

A.7.4 Local and global consistency passes

Local consistency pass. After each execution step, the local pass inspects only the edited neighborhood: newly added objects, moved objects, their support surfaces, and immediate collision partners. It resolves support-height offsets, wall flushness errors, and small collisions with the minimum feasible adjustment that preserves the intended action. Objects in the protected sets P_τ and P_τ^T are never moved or deleted by this pass. If a violation cannot be repaired without touching protected content, the issue is surfaced to the Inspector instead of being silently corrected.

Global feasibility pass. After loop termination, a single global pass checks the full scene for residual collisions and out-of-boundary placements. Resolution priority follows the protection hierarchy: protected content first, then target-related objects introduced by the current instruction, then lower-priority unprotected objects. When two unprotected objects conflict, the pass prefers the smaller displacement and preserves already satisfied hard requirements whenever possible. Remaining infeasible cases are kept as failures and counted by the final OOB / Col. metrics rather than hidden by aggressive cleanup.

Re-verification rules. After each execution and local consistency pass, the Inspector re-verifies the current focused requirement, any requirements whose matched objects were edited or repaired, and any previously satisfied requirements that share objects with the edited neighborhood. A degraded tool call does not mark the original predicate as satisfied unless the corresponding verifier later passes. If a consistency pass changes an object bound to a satisfied requirement, that requirement is returned to the verification set before new protection bindings are committed.

B AuthorBench and Evaluation Protocol

This part specifies the benchmark composition, metrics, external verifier, baseline adapters, and human evaluation protocol used for all reported results.

B.1 Metric definitions

Goal fulfillment. For a case with annotated goal checks $\mathcal{G}$, GF is the fraction of satisfied checks:

$$\mathrm{GF} = \frac{1}{|\mathcal{G}|} \sum_{g \in \mathcal{G}} \mathbf{1}[g \text{ satisfied}].$$

We group checks into constraint families: object constraints (existence, absence, count, scale, and rotation), relation constraints, surface constraints (support and wall placement), and zone constraints (functional zones and clear-area requirements). The benchmark annotation covers the same four families for both construction and editing. Macro-GF is the unweighted mean of the reported family GF values. In the main construction table, we additionally report Macro*, the mean of object/relation/zone GF, and Core, the mean of object/relation GF, to separate support-surface difficulty from the rest of compositional satisfaction. In the main editing table, we report Obj./Rel./Surf. and compute Macro over these reported families only, omitting Zone for readability in the current comparison. All-Goal success is the fraction of cases where every annotated goal check is satisfied.

Editing locality. For editing cases, preservation rate (PR) measures the fraction of protected checks that remain valid:

$$\mathrm{PR} = \frac{1}{|\mathcal{P}|} \sum_{p \in \mathcal{P}} \mathbf{1}[p \text{ preserved}].$$

Unintended change rate (UCR) measures accidental modification of non-target initial objects:

$$\mathrm{UCR} = \frac{\#\{\text{changed non-target objects}\}}{\#\{\text{non-target objects}\}}.$$

Physical validity. OOB (out-of-boundary) and Col. (collisions) are the average number of out-of-boundary objects and colliding object pairs, computed from the final scene geometry. For construction, these measure the physical validity of the generated scene. For editing, these measure new physical

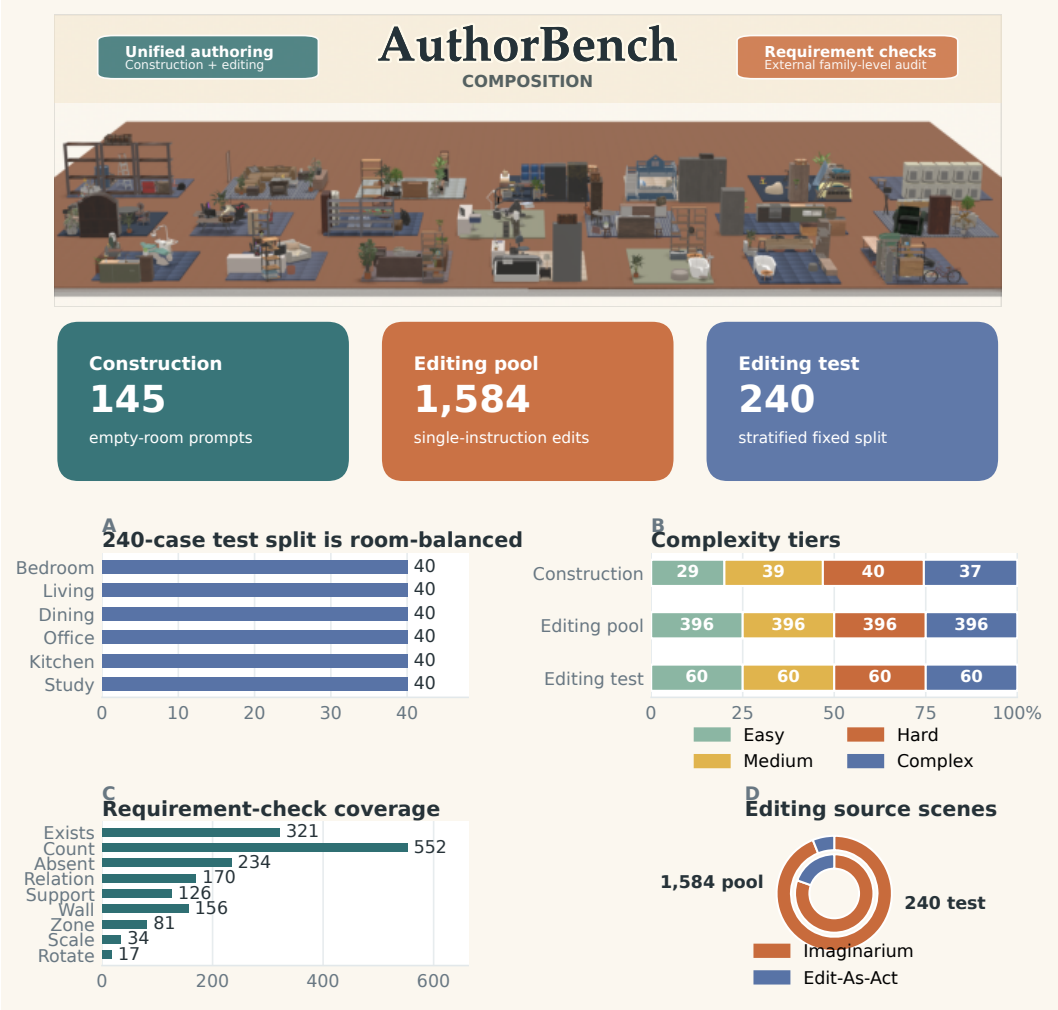


Figure 5: **AuthorBench composition.** The benchmark combines 145 construction cases, a 1584-case editing pool, and a fixed 240-case editing test split. The split is balanced by room type and edit complexity while retaining coverage across source-scene families and check types.

issues introduced by the edit operation, assuming the source scene is physically valid. HCS is a validity-weighted hard-constraint score: the pass rate over existence, absence, count, relation, support, and wall checks, multiplied by $1/(1 + \text{OOB} + \text{Col.})$. It is used only for ablations and diagnostics.

B.2 AuthorBench construction and editing details

This section expands the dataset description in §4.

Subset sizes and tier balance. The construction subset contains 145 cases across six room types: bedroom, living room, dining room, study, and office (25 cases each), and kitchen (20 cases). The editing benchmark consists of a 1584-case instruction pool plus the fixed 240-case test split used throughout this paper. The full editing pool is balanced by edit complexity, with 396 cases in each of E1 atomic, E2 grounded, E3 structured, and E4 compound. The test split is a deterministic stratified sample with 10 cases for each (edit-complexity, room-type) cell, giving 40 editing cases per room type and 60 cases per complexity tier. The released benchmark contains 1043 construction goal checks, 4140 editing-pool goal checks with 4752 preserve checks, and 648 goal checks with 720 preserve checks on the fixed editing test split.

Three-stage construction pipeline. Each case is produced through:

Table 13: **Editing pool and test-split coverage.** (a) summarizes edit tiers and source families; (b) summarizes source-scene complexity and target burden. The fixed test split preserves the main operation families from the 1584-case pool while enforcing room-level balance.

(a) Edit design

Statistic	Pool	Test
E1 atomic	396	60
E2 grounded	396	60
E3 structured	396	60
E4 compound	396	60
Imaginarium	1488	193
Edit-As-Act	96	47

(b) Scene and target burden

Statistic	Pool	Test
S1 simple	300	35
S2 medium	1008	135
S3 dense	192	41
S4 extreme	84	29
Q0 single target	1068	161
Q1 two–three targets	374	57
Q2 four-plus targets	142	22

- *(i) Skeleton drafting.* Per room type, we instantiate skeleton templates parameterized by required object inventory, allowed/forbidden categories, anchor surfaces, wall references, and functional-zone composition. Templates are sampled across the four tiers so that tier statistics are balanced within each room type.
- *(ii) Constraint compilation.* Each skeleton is expanded into a natural-language prompt and an external constraint set. Every constraint carries one of six types (object, relation, surface, zone, preserve, forbid) and, where feasible, a machine-checkable predicate (support-on, wall-anchored-to, count, negative-existence, free-floor area; see Appendix B.3).
- *(iii) Human review.* Two annotators independently verify that the prompt, source scene (for editing), and predicate set are mutually consistent, and that the case is realizable. Disagreements are revised or discarded.

Tier definitions. Easy cases mainly test object inventory, count, and negative constraints. Medium cases add spatial relations and wall placement. Hard cases introduce support-surface reasoning and functional-zone composition. Complex cases mix multiple constraint families in dense compound prompts or edits, increasing the number of interacting goals and protected objects.

Editing source scenes. Editing source scenes are drawn from Edit-As-Act [18] and Imaginarium [16], then normalized into the AuthorBench scene schema. Normalization assigns stable object identifiers, canonical category names, simple geometry, support surfaces, wall anchors, and functional zones, so target grounding ("the desk on the east wall") is unambiguous and protection checks can be verified against original object IDs. The same normalized source scene is provided to every method for a given editing case, so editing scores reflect local modification and preservation rather than differences in upstream scene generation. The benchmark-side external checks are authored from the case skeletons plus human review, rather than copied from MUSE's compiled requirement program, and are not exposed to any method as gold test-time supervision.

Case coverage. Construction cases span object inventory, count, and negative constraints (Easy), spatial and wall-placement (Medium), support-surface and zone composition (Hard), and dense compound prompts mixing all families (Complex). Editing cases include atomic edits (add/move/remove/replace) and compound instructions, with boundary cases that target support-surface management, wall-mounted objects, target grounding, and controlled local modification.

Split balancing and reporting. The fixed construction and editing splits are balanced jointly by room type and difficulty tier, and the editing split is additionally balanced by edit family. These distributions are used only to construct a stable and reviewer-auditable benchmark protocol; they are not exposed to the method at test time. The predicate inventory used by the external checks is listed separately in Appendix B.3.

Independent semantic judging. Deterministic predicates are used whenever a requirement can be expressed over the exported scene graph or geometry. Residual semantic or perceptual constraints are evaluated by an independent judge that receives the original requirement, the exported scene representation, and rendered views under the same prompt and protocol for all methods. The judge is

Table 14: **AuthorBench predicate inventory by requirement family.** External checks are evaluated after scene export and are independent of MUSE's internal verifier. Deterministic checks take precedence whenever available; open-ended residuals are handled by the independent semantic judge.

Family	Predicates / checks	Inputs	Typical failure modes
Object	`exists`, `count`, `negative-existence`, category match	Object IDs, canonical category, count target	Missing object, extra forbidden object, wrong category
Relation	`left_of`, `right_of`, `front_of`, `behind`, `near`, `far`, `parallel_to`	Subject/object boxes, room frame, facing direction	Wrong side, too far/near, orientation mismatch
Surface	`on_top_of`, `under_support`, `wall-anchored-to`, `against_wall`	Support surface, wall domain, object footprint, height	Unsupported object, wall offset, wrong support
Zone	`inside_zone`, `zone_contains`, `free-floor-area`, clear functional region	Zone polygon, object footprint, free-space grid	Object outside zone, cluttered zone, insufficient clear area
Preserve	`preserve_existence`, `preserve_pose`, `preserve_relation`, `preserve_support`	Source-scene IDs, original poses and relations	Deleted non-target, moved anchor, broken support/relation
Forbid	`forbid_category`, `forbid_region`, `forbid_relation`	Negative constraint, scene objects, zones/relations	Forbidden object, forbidden placement, forbidden relation
Physical	OOB, collision pairs, support-height consistency	Room boundary, bounding boxes, support heights	Out-of-room object, interpenetration, floating/sunken object

reserved for non-programmable constraints and never overrides deterministic predicate outcomes, preserving the separation between AuthorBench evaluation and any method-internal verifier. This means AuthorBench is benchmark-side and method-agnostic: the same checking protocol is applied to all exported scenes regardless of whether a method internally uses requirement programs, action plans, or holistic generation.

B.3 External verifier predicate catalog

The rule-based hard-requirement verifier implements deterministic geometric checks over typed `VerifierConstraint` records with explicit subject and object references. In practice, it covers entity predicates such as `count` and `exists`; relative-placement predicates such as `left_of`, `right_of`, `front_of`, `behind`, `near`, `far`, and `parallel_to`; support and wall checks such as `on_top_of`, `under_support`, and `against_wall`; as well as zone, preserve, forbid, and physical-validity checks summarized in Table 14.

For each predicate, the verifier also produces structured diagnostics:

- `failure_mode`: Why the check failed (e.g., `wrong_side`, `too_far`, `count_mismatch`).
- `repair_hint`: Suggested corrective action (e.g., "move subject 0.3m to the left").
- `matched_ids`: Object IDs that participated in the match (enabling protection set expansion on satisfaction).
- `metrics`: Quantitative measurements (e.g., actual distance, actual count).

B.4 Verifier coverage and fallback judgment

The external AuthorBench checks are independent from MUSE's internal verifier. MUSE uses requirement-level verification inside the loop to decide what to do next, but all reported paper metrics are computed from the external benchmark checks after scene export. This separation prevents the system from being rewarded for satisfying only its own internal judgment interface.

In practice, the rule-based verifier covers grounded count, existence, relation, support, wall, and zone-like checks whenever the requirement can be expressed as a typed predicate over the scene graph. Model-assisted judgment is reserved for residual style or open-ended semantic feedback that is difficult to encode as a deterministic geometric rule. When the internal verifier cannot certify a

Table 15: **Baseline interfaces and AuthorBench adapters.** Each baseline keeps its released backbone and native control interface; the adapter only standardizes the input scene format and final exported scene for external AuthorBench evaluation.

Baseline	Native interface	AuthorBench adapter
LayoutGPT / LayoutGPT-E	One-shot text-to-layout prediction	Prompt plus room/source-scene serialization to unified scene JSON
LayoutVLM	Construction-only layout optimization	Empty-room shell plus asset metadata to unified scene JSON
Holodeck	Construction-only single-room generation pipeline	Empty-room shell plus AuthorBench prompt to unified scene JSON
ReSpace / ReSpace-E	Structured scene representation with native edit pipeline	Convert released SSR scene once into AuthorBench scene JSON
Edit-As-Act	Goal-regressive editing action loop	Initial scene plus instruction through prepared action interface
SceneReVis	Multi-turn language-guided revision loop	Released batch pipeline with exported scene snapshots

Table 16: **Baseline limitations and failure handling.** Unsupported / weak families are constraint families that are not explicitly represented or verified in the baseline's native interface. Failed or partial runs are scored from the last emitted scene rather than manually repaired.

Baseline	Unsupported / weak families	Failure handling
LayoutGPT / LayoutGPT-E	No native protection-state mechanism; zone constraints are weak; support/wall constraints are prompt-only	Score the converted final layout without manual repair
LayoutVLM	No editing interface; no native preservation mechanism; support/zone constraints are weak	Score the exported final layout as-is
Holodeck	No editing interface; no native preservation mechanism; dense support/wall constraints are weak	Score the exported final scene as-is
ReSpace / ReSpace-E	Support/wall and zone constraints are weak; editing is preservation-biased	Score the converted final SSR scene without manual completion
Edit-As-Act	No construction interface; surface- and zone-heavy edits are weak	Score the last emitted final scene without retries or repair
SceneReVis	No explicit protection sets; support/wall verification is weak under the native interface	Score the exported final scene, or the last emitted snapshot if the run is partial

requirement, it may still provide guidance to the planner, but the benchmark score always comes solely from the external checks defined by AuthorBench. The external predicate catalog is method-agnostic: any baseline or future method that exports an evaluable scene in the unified AuthorBench schema can be scored by the same post-export checker, without requiring it to share MUSE's internal decomposition or verifier interface.

B.5 Baseline adapters and fairness protocol

Fairness protocol. We keep each baseline's released backbone, native prompting style, and stopping rule, and standardize what is shared across methods: the AuthorBench case definitions, room shells or source scenes, the common asset pool when a conversion step is required, and the final rendering and evaluation stack. When a baseline emits a native layout or scene format, we convert it once into the unified AuthorBench scene JSON and then score it with the same verifier as MUSE; failed or partial runs are counted from the method's last emitted scene rather than manually repaired. The comparison is therefore between released end-to-end systems under a shared benchmark protocol, not a counterfactual setting where MUSE-specific verification, repair, or protection modules are injected into baselines. Fairness comes from shared inputs, normalization, export format, and external scoring, while each method's internal control mechanism remains unchanged.

LayoutGPT / LayoutGPT-E. LayoutGPT is used as a one-shot LLM layout baseline. For construction, it receives the AuthorBench prompt and empty room shell; for editing, we serialize the provided source scene and edit instruction into the adapter format used for the 240-case editing split.

In both cases, the predicted categories and placements are converted to the unified scene JSON, then rendered and scored with the same downstream pipeline as the other methods.

LayoutVLM. LayoutVLM is evaluated on construction using the same room shell and prompt, together with the candidate asset metadata required by its optimization-based interface. We run its native layout optimization once per case, export the resulting placement JSON to the AuthorBench scene format, and apply no method-specific corrective post-processing beyond the common renderer and evaluator.

Holodeck. Holodeck is evaluated on construction only. We keep its native single-room generation pipeline and provide the same AuthorBench prompts and empty room shells, then export the generated final scene to the common JSON format for unified rendering and scoring. This preserves Holodeck's own object-selection and placement behavior while avoiding any per-case retuning on our side.

ReSpace / ReSpace-E. ReSpace is evaluated through its native structured scene representation (SSR) and asset-sampling pipeline. For construction, it starts from the same room boundary; for editing, it receives the same initial scene and instruction and performs its released prompt handling and edit execution before exporting a final SSR scene. We then convert that scene once into the AuthorBench format and evaluate it with the shared verifier.

Edit-As-Act. Edit-As-Act is evaluated on the 240-case editing test split only. We provide the same initial scene and user instruction through a prepared action interface, run its standard action loop to termination, and record the resulting final scene without manual intervention or case-specific retries. The final structured scene is then mapped into the common evaluation schema.

SceneReVis. SceneReVis is run with its released AuthorBench batch pipeline and native multi-turn revision loop. We preserve its default iterative behavior and evaluate the exported final scene when available, or the last emitted scene snapshot when a run stops with only a partial scene, so unsuccessful trajectories remain part of the reported averages. As with the other baselines, the exported scenes are rendered and scored through the common AuthorBench pipeline.

B.6 Human evaluation interface

This section shows the questionnaire interfaces used for our human evaluation setup. The first interface presents the source scene and the edited result side by side, then asks raters to score the edited output along instruction satisfaction, preservation, physical plausibility, and overall preference. The second interface presents four candidate outputs for the same editing case and asks raters to make an overall forced-choice decision.

C Additional Experimental Results

C.1 Additional qualitative results

This section supplements the main-paper qualitative comparison in Figure 4 with additional construction and editing cases from AuthorBench.

C.2 Continuous editing stress test

We further evaluate MUSE in a continuous editing stress test that extends beyond isolated single-instruction edits. The test contains 50 three-stage editing chains, for 150 stage-level tasks in total. In each chain, stage 1 constructs the initial scene, while stages 2 and 3 apply local edits to the previous stage's output. This yields 50 initial construction tasks and 100 continuous editing tasks. Unlike the single-instruction AuthorBench editing split, this setting stresses cross-turn state inheritance: later edits must operate on previously generated content while preserving already established scene structure.

Across all 150 stage-level tasks, MUSE completes 139 stages (92.7%). Restricting evaluation to the 100 continuous editing stages, it completes 89 edits (89.0%). At the chain level, 39 of the 50

(a) Result-level rating form. **(b) Four-way forced-choice form.**

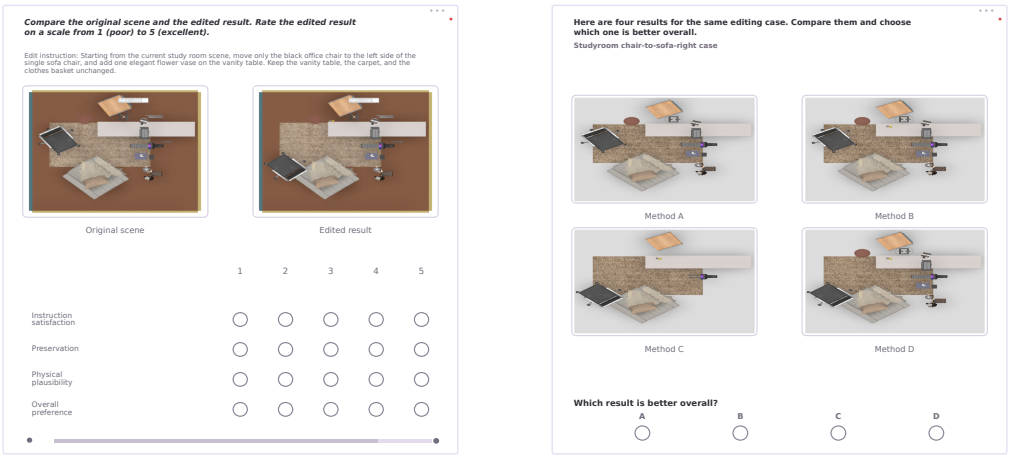


Figure 6: **Human evaluation interfaces for preservation-aware editing.** Panel (a) asks annotators to compare the original scene and the edited result, then rate the edited output on instruction satisfaction, preservation, physical plausibility, and overall preference. Panel (b) presents four candidate results for the same editing case and asks annotators to select the overall best result.

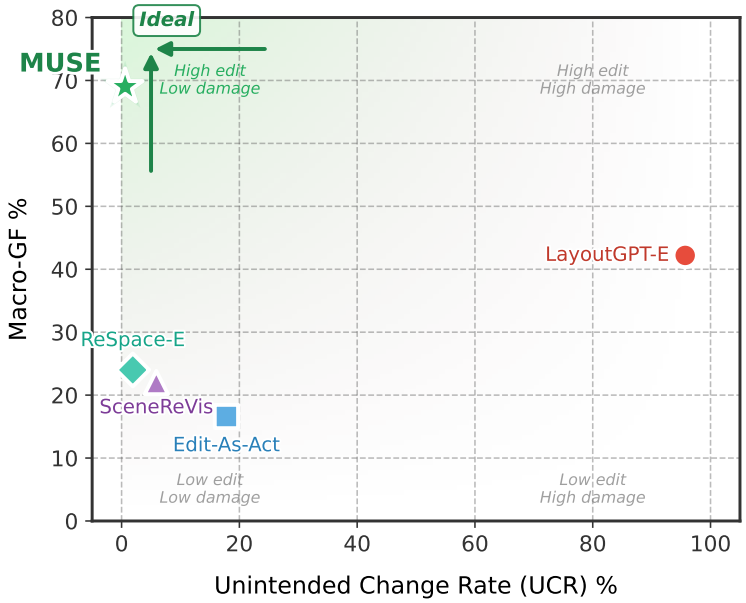


Figure 7: **Completion-locality trade-off.** Higher Macro-GF with lower UCR is preferred. MUSE improves completion while remaining in the low-unintended-change region.

editing chains complete both edit stages successfully. The operation breakdown in Table 17 shows that add and scale operations are robust in this setting, while rotate, replace, and move remain harder because they require precise target grounding and fine-grained geometric control. For rotation, the auxiliary keyword check passes in 8 of 9 cases, indicating that the intended target is usually identified; however, only 4 of 9 satisfy the stricter completion criterion because the verifier still detects unresolved facing-relation errors.

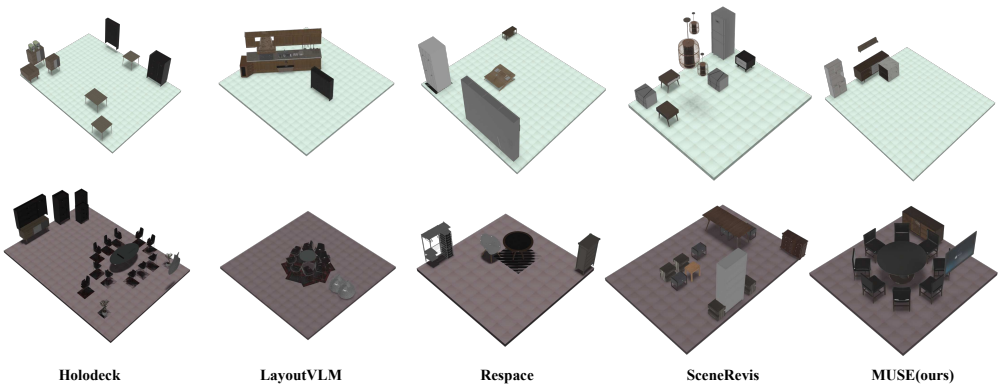


Figure 8: **Additional construction comparisons.** These examples further compare scene construction quality across methods under multi-object spatial and support constraints. MUSE better satisfies compositional requirements while maintaining coherent room layouts.

Table 17: **Continuous editing stress test.** The test contains 50 three-stage editing chains. We report strict completion from the run status and a separate automatic keyword check used only as a diagnostic signal; the latter does not replace requirement-level verification. Rotation has high keyword-level target coverage but lower strict completion because several cases still fail the precise facing-relation check.

Edit family	Tasks	Strict comp.	Strict rate (%)	Keyword check	Typical residual issue
Add	58	57	98.3	51	Occasional local insertion stall or keyword mismatch
Delete	8	7	87.5	6	Target removal can stall without an effective scene change
Replace	8	6	75.0	6	Replacement or the follow-up local adjustment remains incomplete
Scale	8	8	100.0	7	Mostly stable under this stress setting
Move	9	7	77.8	8	Target grounding often succeeds, but the final relation may remain unmet
Rotate	9	4	44.4	8	Target grounding often succeeds, but precise facing orientation remains difficult
Overall	100	89	89.0	86	Failures concentrate on fine-grained local transforms

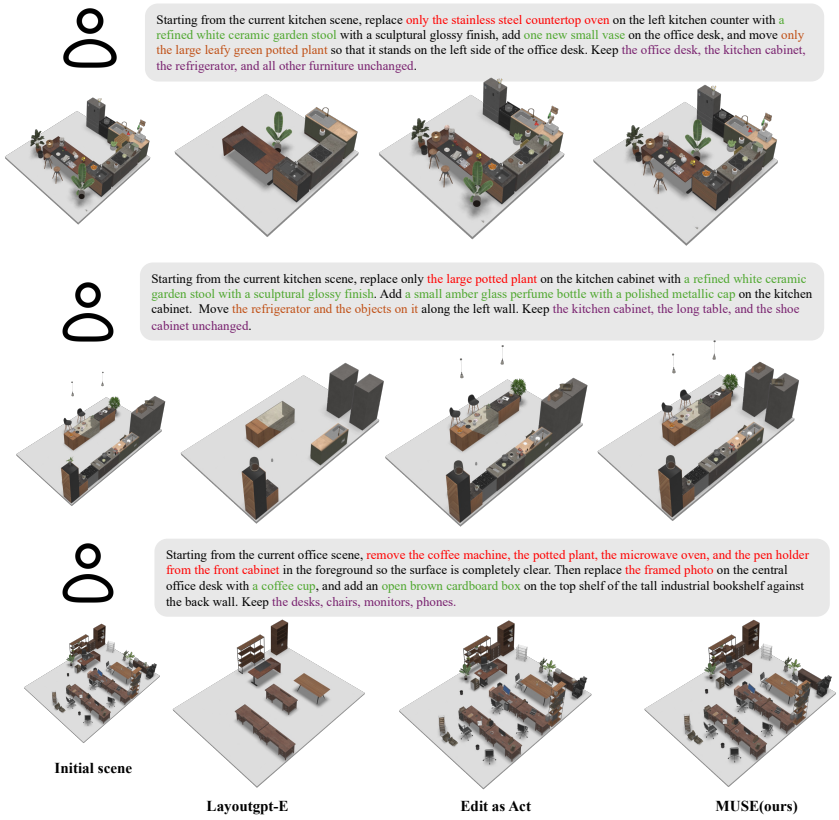


Figure 9: **Additional preservation-aware editing comparisons.** These kitchen and office examples require coupled replacement, insertion, movement, or removal while keeping non-target content stable. MUSE better maintains preservation constraints while completing the requested local changes.

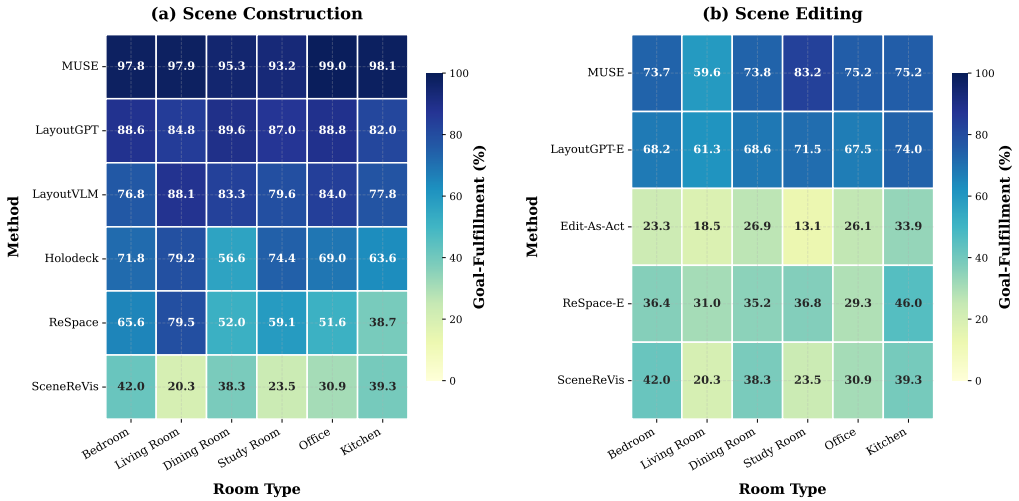


Figure 10: **Goal-Fulfillment by room type (heatmap).** The same per-room-type GF values rendered as heatmaps for quick visual comparison. Darker cells indicate higher scores. The left panel shows construction methods; the right panel shows editing methods. MUSE shows consistently dark rows, indicating robust performance regardless of room type.

C.3 Per-room-type performance breakdown

Construction patterns. All methods perform best on bedroom and living room scenes, where object categories are more standardized and spatial constraints are fewer. Kitchen scenes prove most challenging due to the high density of support-surface requirements and wall-anchored appliances. MUSE maintains the most consistent performance across room types, with the smallest gap between its best and worst room types.

Editing patterns. Editing shows higher variance across room types than construction, reflecting differences in source-scene complexity and edit difficulty. MUSE achieves the best or second-best GF in every room type while maintaining the preservation advantages reported in the main paper.

C.4 Out-of-distribution qualitative visualization

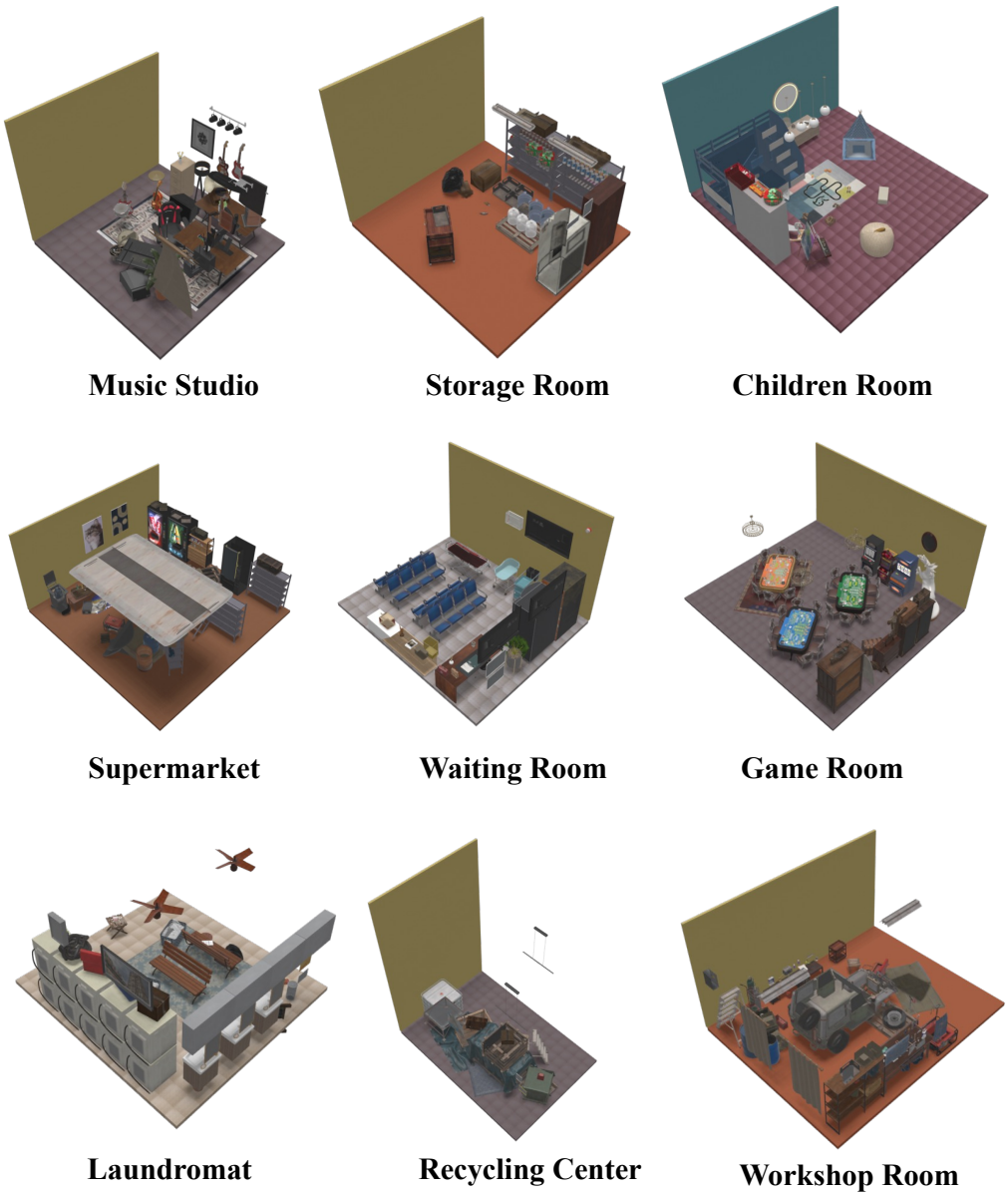


Figure 11: **Out-of-distribution qualitative visualization.** Additional full-page examples illustrating MUSE's behavior beyond the in-distribution benchmark cases discussed in the main paper and earlier appendix sections.

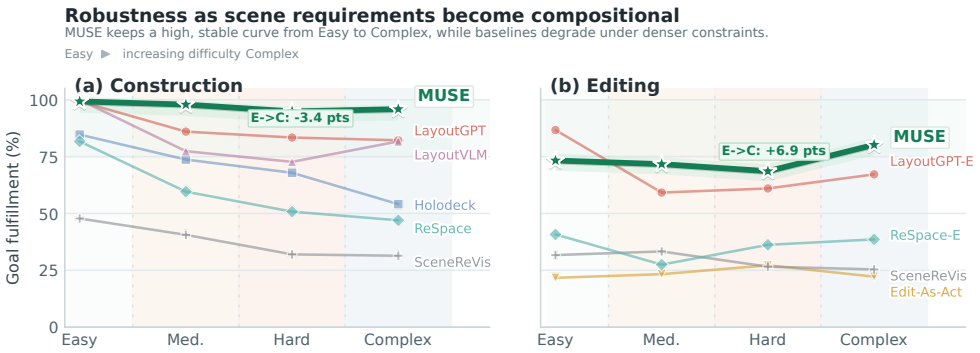


Figure 12: **Complexity robustness across difficulty tiers.** We plot goal fulfillment for each method from Easy to Complex cases in construction and preservation-aware editing. Higher and flatter curves indicate better robustness as the number and interaction of requirements increase.

Table 18: **Navigation curriculum evaluation protocol.** Metrics are computed from exported layouts, not from a trained embodied agent policy.

Parameter / metric	**Definition**
Cases and stages	100 curriculum cases, each with 3 stages, for 300 scenes per method
Grid resolution	0.1 m occupancy grid over the room floor
Obstacle buffer	0.05 m expansion around object footprints
Walkable ratio	Fraction of floor cells not occupied by buffered obstacles
Reachable-object ratio	Fraction of target objects reachable from free space by grid search
Mean access path length	Mean shortest path length from accessible free cells to target access rings
Difficulty monotonicity	Case passes if walkable area does not increase and path length does not decrease beyond tolerance across stages
Free-space preservation IoU	Intersection-over-union of free-space masks between successive stages
Physical validity	Collision and out-of-boundary rates from exported scene geometry

C.5 Complexity robustness

Construction and editing cases are organized into four tiers: Easy, Medium, Hard, and Complex. We report GF for each tier, then compute retention:

$$\text{Ret.} = \frac{\text{GF}_{Complex}}{\text{GF}_{Easy}}.$$

Higher retention indicates better robustness as compositional complexity increases. A retention above 100% indicates that the method performs better on Complex cases than on Easy cases.

C.6 Downstream navigation curriculum details

The downstream curriculum experiment evaluates whether a sequence of related scenes remains spatially coherent as task difficulty increases. Each of the 100 cases contains three stages in the same room. The incremental variant edits the previous stage with MUSE; the regeneration variant creates each stage independently from the corresponding stage prompt. Both variants are exported to the same scene format and evaluated with the same grid-based navigation proxy.

C.7 Runtime, API cost, and export availability

All reported results are inference-only: we do not train or fine-tune on AuthorBench. Experiments were run on internal GPU servers for model execution, together with CPU-side scene conversion, rendering, and rule-based checking. The main cost comes from repeated planner / verifier calls, scene export, and final evaluation across the 145 construction cases, the fixed 240-case editing split, and the ablation settings.

Table 19: **Full stage-wise navigation curriculum metrics.** Incremental MUSE and stage-wise regeneration both produce navigable scenes, but incremental editing better preserves free space and difficulty progression across stages.

Method	Stage	Walkable	Reachable	Path m	Col.	OOB
Incremental MUSE	1	0.906	1.000	1.235	0.000	0.000
Incremental MUSE	2	0.886	1.000	1.554	0.000	0.000
Incremental MUSE	3	0.868	0.992	1.736	0.000	0.000
Stage-wise regeneration	1	0.912	1.000	1.233	0.000	0.000
Stage-wise regeneration	2	0.894	1.000	1.794	0.000	0.000
Stage-wise regeneration	3	0.878	0.999	2.183	0.000	0.000

Method	Avg LLM calls	Avg turns	Avg time / case	Est. API cost / case
Simple cases (easy)				
Edit-As-Act	$\sim$1	1	$\sim$45s	$\sim$\$0.04
LayoutGPT-E	$\sim$1	1	$\sim$30s	$\sim$\$0.05
MUSE	3.87	0.82	260s	\$0.147
Hard cases (hard + complex)				
Edit-As-Act	$\sim$1	1	$\sim$90s	$\sim$\$0.08
LayoutGPT-E	$\sim$1	1	$\sim$60s	$\sim$\$0.09
MUSE	9.10	3.27	574s	\$0.426

Table 20: **Reference efficiency profile for selected editing methods.** MUSE values are measured from editing-run logs on the 240-case AuthorBench split. Edit-As-Act and LayoutGPT-E are reference estimates under stateless single-turn adaptation and should be interpreted as approximate efficiency profiles rather than exact billing logs. We omit ReSpace-E and SceneReVis from this compact table because their released pipelines expose less directly comparable runtime logging and status granularity; output availability for all editing methods is reported separately in Table 21. Estimated API cost reflects text-only prompt/response usage and may not fully account for image-token charges.

Table 20 provides a compact reference efficiency profile for the editing setting. We report a selected subset of methods with directly comparable single-turn or iterative call structure and sufficiently stable per-case logging, so the table stays readable and focuses on the main runtime trend. We group cases into simple edits (the easy tier) and hard edits (the union of the hard and complex tiers).

Wall-clock time varies across baselines because they expose different released pipelines and different numbers of model calls. We therefore report unified evaluation outcomes rather than a single cross-method runtime claim. Table 20 should be read as a cost-profile comparison: single-turn baselines stay cheap because they typically make one stateless edit call, whereas MUSE spends more calls and more repair turns to support iterative grounding, verification, and correction. The total project compute exceeded the final reported runs because prompt iteration, dataset curation, debugging, and pilot evaluations were performed before freezing the benchmark protocol and final tables.

Because released baselines expose heterogeneous runtime status labels, we also report a simple output-availability metric: whether a case emits an evaluable `final_scene.json` in the common AuthorBench format. This *final-scene export rate* measures generation availability rather than requirement satisfaction, and therefore complements rather than replaces the All-Goal success values in the main paper.

D Implementation, Assets, and Prompts

D.1 Requirement enrichment

During requirement program compilation, the Architect automatically detects and resolves incomplete dependency chains. A requirement r_i with a placement constraint (e.g., "place a lamp on the nightstand") implicitly depends on the existence of the referenced object (the nightstand). If no `hard` or `edit` requirement in the current program declares that object, the system injects an implicit entity requirement:

Task	Method	Final scenes	Export rate
Construction	LayoutGPT	145/145	100.0%
Construction	Holodeck	145/145	100.0%
Construction	LayoutVLM	141/145	97.2%
Construction	ReSpace	145/145	100.0%
Construction	SceneReVis	99/145	68.3%
Construction	MUSE	145/145	100.0%
Editing	LayoutGPT-E	240/240	100.0%
Editing	Edit-As-Act	212/240	88.3%
Editing	ReSpace-E	240/240	100.0%
Editing	SceneReVis	229/240	95.4%
Editing	MUSE	240/240	100.0%

Table 21: **Final-scene export rate on AuthorBench.** A case counts as export-successful if the released pipeline emits an evaluable `final_scene.json` in the common AuthorBench format. This is an output-availability metric, not a task-success metric: a run may export a final scene even when it stops early or fails to satisfy all requirements.

Table 22: **Representative motif cards.** Cards are compact priors used as structured context for requirement compilation; they are not additional learned model parameters.

Motif	**Triggers**	**Decomposition prior**	**Negative hint**
`bed_setup`	bedside, flanking the bed	Split left/right nightstands and lamps into separate relation/support checks	Do not use one object for both sides
`dining_set`	chairs around table, dining area	Bind table first, then place chairs around distinct sides or perimeter slots	Do not collapse all chairs into count only
`tv_wall`	TV wall, media console	Ground wall direction, place console on floor, attach TV to same wall	Do not treat TV as freestanding if wall-mounted
`reading_corner`	reading nook, cozy corner	Create zone, chair, side table, lamp, and clear access relation	Do not scatter objects across unrelated zones

1. For each requirement r_i with a verifier constraint ϕ_i, extract the *object* reference (if any).
2. Check whether any existing requirement in $\mathcal{B}_t$ already ensures the referenced object's existence (via a `count` or `exists` predicate targeting the same `canonical_type`).
3. If not, inject a new `hard` requirement with predicate `exists` and the referenced object's type, assigned a higher priority than r_i to ensure it is processed first.
4. Update r_i's `depends_on_requirement_ids` to reference the injected requirement.

This enrichment ensures that the focus scheduler processes entity requirements before their dependent placement requirements, preventing "dangling reference" failures where the planner attempts to place an object relative to something that does not yet exist.

D.2 Skill memory retrieval and library maintenance

D.2.1 Motif card schema

Each motif card in skill memory $\mathcal{K}$ stores a compact pattern prior: a `motif_type`, trigger phrases that activate it, expected anchor objects, and a small guidance payload covering preferred decomposition, role bindings, optional qualifiers, and negative hints. Cards may also specify an `exclusive_with` group so retrieval keeps only one competing interpretation. For example, a `bed_setup` card can be triggered by phrases such as `["nightstand on each side", "bedside", "flanking the bed"]` and can recommend splitting the request into separate left/right requirements around a shared bed anchor.

D.2.2 Retrieval mechanism

Card retrieval proceeds as follows:

1. For each card in $\mathcal{K}$, compute a relevance score based on trigger-phrase overlap (exact match, substring, and keyword overlap) with the input instruction I_t and anchor-pattern overlap with the current scene objects.
2. Rank cards by score and select the top-k (k=2 by default).
3. Apply mutual-exclusion filtering: if two selected cards are in the same exclusive group, retain only the higher-scoring one.
4. Inject the selected cards' `preferred_decomposition` and `negative_hints` into the Architect's prompt as structured context.

D.2.3 Offline library expansion protocol

Outside benchmark evaluation, successful authoring episodes can be mined offline to expand the motif-card library. Starting from a set of verified trajectories, the library builder extracts candidate motif cards as follows:

1. Group the satisfied requirements by their spatial scope (zone-level groupings).
2. For each group with ≥ 2 requirements, extract the decomposition pattern: the set of predicate types, role labels, and spatial relations used.
3. Match the extracted pattern against existing cards in $\mathcal{K}$. If a match is found (same motif type and anchor pattern), update the card's trigger phrases and qualifiers. If no match, create a new candidate card.
4. De-duplicate and persist the resulting library before the next evaluation or deployment cycle.

This protocol ensures that skill memory grows conservatively (only from verified successes) and avoids introducing noisy priors from failed or partial authoring episodes. In all experiments reported in this paper, the resulting library is frozen at test time.

D.3 Asset library and retrieval details

MUSE decouples *what* object should be instantiated from *where* it should be placed. The Architect and Sculptor therefore emit object-centric descriptions and size hints, while spatial grounding is handled by the placement tools and verifier loop rather than being baked into asset retrieval text. This separation prevents the retriever from conflating appearance and category matching with downstream geometric constraints such as support placement, wall attachment, or preservation-aware local edits.

D.3.1 Unified asset registry

Our implementation maintains a unified asset registry that merges the main 3D-FUTURE pool with optional long-tail or dynamic sources into a single searchable catalog. Each registry record stores a stable asset key and source label together with the fields needed for retrieval and later scene export: render path, axis-aligned bounding-box size, a text caption, optional category, descriptive tags, and indices to precomputed text / image embeddings. In practice, these fields cover identity and source filtering (`asset_key`, `asset_id`, `source`), geometric sanity checks (`bbox_xyz`), semantic matching (`caption`, `tags`, optional `category`), embedding lookup, and export bookkeeping. The released registry manifest used in our implementation contains 66,655 records in total, while retaining source labels so that retrieval can be restricted to subsets such as 3D-FUTURE only when a baseline interface requires it.

D.3.2 Query formation and scoring

At execution time, each retrieval query is formed from the object-centric description emitted by the planner (e.g., "compact desk lamp" or "round dining table"), optionally paired with a target size hint from the requirement or edit operator. Spatial instructions such as "on the nightstand", "against the east wall", or "near the sofa" are deliberately not inserted into the retrieval text; they are enforced later by `place_on_support`, `attach_to_wall`, `add_object_with_relation`, or verifier-guided repair. This keeps the retriever focused on identity, appearance, and coarse scale.

For a query q, candidate asset a, and target size $\mathbf{s}^*$, the retriever ranks assets using a weighted score

$$\text{score}(a \mid q, \mathbf{s}^*) = \lambda_{\text{sbert}} \cos(\mathbf{e}_q^{\text{text}}, \mathbf{e}_a^{\text{text}}) + \lambda_{\text{clip}} \max_v \cos(\mathbf{e}_q^{\text{clip}}, \mathbf{e}_{a,v}^{\text{clip}}) - \lambda_{\text{size}} \cdot \frac{1}{3} \sum_{d \in \{x,y,z\}} \frac{|s_{a,d} - s_d^*|}{\max(s_d^*, \epsilon)}.$$

The first term matches the query against normalized SBERT text embeddings, the second term adds multiview CLIP similarity when image features are enabled, and the last term penalizes size mismatch with the requested bounding-box hint. Source filters are applied before top-k extraction, and the resulting candidate list is sorted by the final score.

D.3.3 Selection policy and traceability

By default, MUSE retrieves the top-k candidates with k=20. Deterministic evaluation uses greedy top-1 selection after ranking, while stochastic variants may sample within the retrieved set using the normalized final scores. After an asset is selected, the scene object stores the chosen `asset_id`, `asset_source`, retrieved bounding box, render path, and a `selection_breakdown` containing the SBERT score, CLIP score, size error, and final combined score. This makes asset choices inspectable during debugging and lets the verifier distinguish retrieval mistakes from downstream placement mistakes.

D.4 Prompt template examples

The production prompts used in our implementation are long because they explicitly enumerate output schemas, allowed predicates, tool signatures, and safety constraints. Since reproducibility is easier to assess when reviewers can inspect concrete prompt wording, we include one representative excerpt below and summarize the remaining agent contracts in Table 23. To keep the appendix readable, we omit repetitive enum expansions and large JSON schema dumps, but the excerpt below matches the core wording and rule ordering of the prompt used in code.

Architect system prompt excerpt.

```
You are the DesignBrief parser in the MUSE scene authoring system.  Your
only job is to convert a user's one-shot high-level request into a
structured DesignBrief JSON object.  Output exactly one valid JSON
object.  Do not output Markdown.  Do not output any explanation.

Core rules.  (1) original_request must preserve the user's request
exactly.  (2) Split the request into trackable atomic constraints and
assign sequential ids req_1, req_2, ....  (3) Use requirement types hard,
forbid, preserve, and edit; move style, material, color, mood, and
density preferences into top-level design_guidance and optional
style_profile.  (4) Requirement text must be faithful, concise, and
written in English even if the input language is not English.  (5) Do
not invent objects, styles, or hidden constraints.

Verifier-friendly normalization.  When a hard requirement can be
normalized, prefer forms such as include <count> <object>, <A> near <B>,
<A> on <B>, <A> under <B>, <A> against wall, left/right/front/behind, or
parallel to.  Normalize faithful synonyms such as next to → near,
literal support placement → on_top_of, and literal under-support
placement → under_support.  If a clause is still not verifier-safe
after decomposition, keep it as plain text with verifier_constraint =
null instead of distorting it.

Memory-guided parsing.  If retrieved parser memory cards clearly match,
use them as interpretation guidance for decomposition, semantic-role
binding, and useful qualifiers, but do not copy them verbatim and do not
let them override the user's explicit request.  If a user preference
profile is provided, treat it only as a soft bias when the request is
ambiguous.
```

Other agent contracts. The Sculptor and Inspector prompts follow the same contract style but specialize it to tool planning and requirement-level judgment. Rather than reproducing two more long prompt blocks, Table 23 lists the concrete behavior that remains exposed in the appendix for each agent.

Runtime assembly. The full runtime message sent to each model call contains more than the system prompt excerpt shown above. For the Architect, MUSE prepends few-shot examples and may inject retrieved motif cards plus a user preference profile. For the Sculptor, the user message serializes

Table 23: **Prompt-contract coverage in the appendix.** We expose one representative prompt excerpt directly and summarize the remaining agent-specific behavioral contracts here, leaving bulky schemas and serialized runtime state to the released code and artifacts.

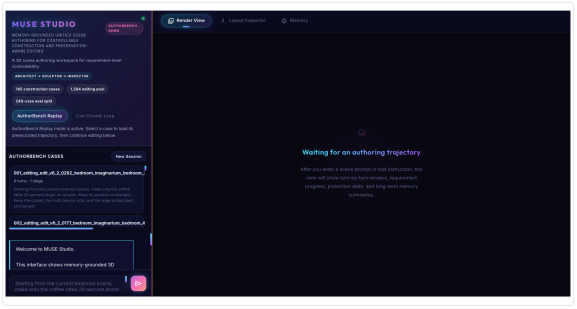
Agent	Primary output	Prompt details shown here
Architect	`DesignBrief` JSON	Exact output discipline, atomic decomposition rules, verifier normalization policy, memory-guided parsing rules
Sculptor	`<tool_calls>` JSON block	Protection constraints, high-level tool preference, local repair policy, anti-repeat and scope-locality rules
Inspector	feedback JSON	Per-requirement judging contract, issue emission rules, termination criteria, model-judged verification policy

the current focus requirement, pending same-scope requirements, protection sets, recent actions, active issues, and scene summaries. For the Inspector, the user message serializes the requirement list, execution result, and current scene, and additionally attaches the rendered image when available. Thus, the appendix excerpt above captures the stable *agent contract* rather than the full runtime prompt; the runtime instance is formed by pairing that contract with structured scene-specific context.

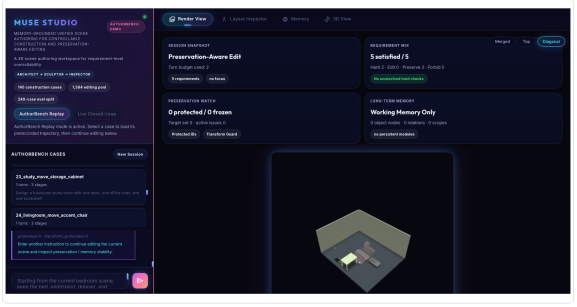
D.5 Interactive frontend

Beyond offline benchmark export, MUSE is also exposed through an interactive frontend for replaying AuthorBench cases and inspecting intermediate system state. Figure 13 shows the main interface organization: an initial entry state, the replay workspace after loading a case, the linked 2D layout inspector, and the memory / verification panel used to surface requirement progress and protection state.

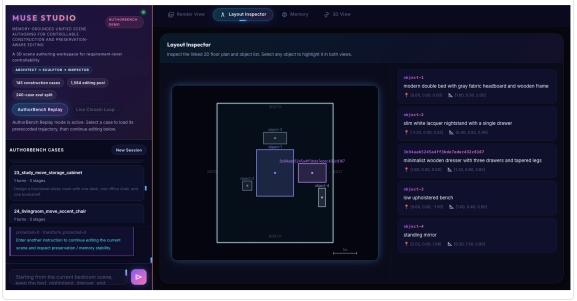
(a) Initial interface



(b) Loaded replay case in Render View



(c) Layout Inspector



(d) Memory and verification panel

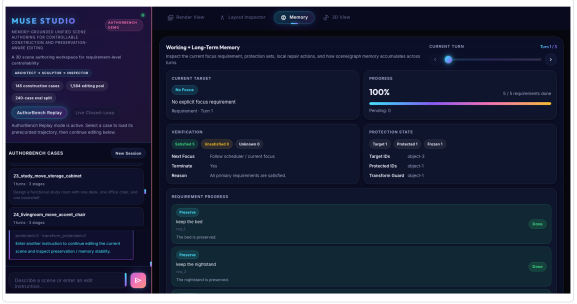


Figure 13: **Interactive frontend of MUSE Studio.** The interface supports AuthorBench case replay and exposes four complementary views: the initial entry state, the main render workspace for a loaded case, the linked layout inspector with object-level geometry, and the memory / verification panel that surfaces requirement progress, protection sets, and working-memory state.

E Limitations and Broader Impacts

E.1 Failure mode analysis

We summarize the main failure modes observed in incomplete or partially completed runs. First, fine-grained local transforms remain the most fragile operation family. In these cases, the system usually preserves the existing scene and often grounds the target object, but the final movement, scale, or orientation constraint can remain unsatisfied after repeated repair attempts. Rotation is the clearest example: several runs pass coarse target checks but fail the verifier because the object does not precisely face the requested reference.

Second, support-surface and wall-grounding constraints are difficult when they interact with room layout constraints. Construction cases that combine object inventory, support placement, wall anchoring, and zone composition can fail even when the requested object categories are mostly present, because satisfying all spatial predicates jointly requires precise placement and later feasibility repair. Third, compound edits are more error-prone than atomic edits. A replacement or insertion may succeed at the object level, while a dependent relation such as "near", "left of", "on top of", or "facing" remains unresolved. Finally, a small number of cases fail during residual physical cleanup, showing that semantic satisfaction and geometric feasibility are not always aligned in dense scenes. These patterns indicate that persistent scene state improves local continuity, but high-precision geometric control and multi-constraint reconciliation remain important directions for future work.

E.2 Limitations

MUSE has several limitations. First, the main benchmark focuses on indoor scenes and single-instruction edits; the supplementary continuous-editing stress test only covers controlled three-stage chains, so the current results do not establish performance on outdoor assets, open-ended multi-turn authoring sessions, or substantially different scene ontologies. Second, MUSE relies on repeated LLM planning and verification calls; when the model misunderstands a requirement or proposes a poor intermediate action, the loop can incur extra latency or terminate with only partial progress. Third, the pipeline assumes access to a structured scene representation and a sufficiently rich asset pool, which may not hold in settings with noisy geometry, missing metadata, or sparse object libraries. These limitations motivate broader-domain benchmarks and more efficient verification and memory mechanisms.

E.3 Broader impacts

Controllable 3D scene authoring can benefit embodied-AI simulation, interactive environment design, and assistive layout tools by making requirements explicit and auditable. At the same time, systems of this kind can produce unsafe, unrealistic, or misleading layouts if they are used without human review or if downstream users over-trust partially satisfied outputs.

Our mitigation emphasis is on verifiability rather than autonomy. MUSE exposes requirement-level checks, reports preservation and unintended-change metrics for editing, and is evaluated with external benchmark checks that are independent from the system's internal verifier. Even so, human oversight remains necessary when generated scenes are used for training, design, or any downstream decision-making setting.